BPSC TRE 4.0

Computer Science (Class 11-12)

Day 9

I. Computer Fundamentals > Boolean Logic & Digital Circuits

2026-11-05

BPSC TRE 4.0 Computer Science — Day 9 — I. Computer Fundamentals > Boolean Logic & Digital Circuits — 2026-11-05

Notes • 44 Questions • Answer Key + व्याख्या

Part 1/3 Digital Basics

(1) 1. आसान भाषा में

- Computer के अंदर सारा काम बिजली के दो स्तरों (levels) से होता है: बिजली है (HIGH) या नहीं है (LOW). हम इन्हें 1 और 0 लिखते हैं. इसे Digital (अंकीय) system कहते हैं.
- Analog system में signal लगातार बदलता है (जैसे पुराने रेडियो की आवाज़). Digital में सिर्फ दो ही स्थितियाँ होती हैं, इसलिए शोर (noise) से कम बिगड़ता है और तेज़ चलता है.
- Boolean algebra वह गणित है जिसमें variable की value सिर्फ 0 या 1 होती है. इसे George Boole ने बनाया. इसी से computer के सारे circuits (logic gates) बनते और सरल किये जाते हैं.

(2) 2. रोज़मर्रा का example

- घर का बल्ब और switch: switch ON (1) तो बल्ब जलता है, OFF (0) तो बुझा रहता है. बीच की कोई स्थिति नहीं. यही digital है.
- दो switch एक के पीछे एक (series) लगाओ तो बल्ब तभी जलेगा जब दोनों ON हों, यह AND है. दो switch अलग-अलग रास्ते (parallel) में लगाओ तो एक भी ON हो तो बल्ब जलेगा, यह OR है.

(3) 3. ज़रूरी शब्द (हर शब्द का मतलब)

- Bit = Binary Digit, यानी 0 या 1.
- Truth table = वह table जिसमें inputs की सारी combinations और उनका output लिखा होता है.
- Rows की संख्या: n inputs हों तो truth table में 2^n rows (inputs 1 -> 2 rows, 2 -> 4, 3 -> 8, 4 -> 16).
- कुल Boolean functions: n variables के लिए 2^(2^n). 2 variables के 2^4 = 16 functions, 3 variables के 2^8 = 256.
- Combinational circuit = output सिर्फ अभी के inputs पर निर्भर (जैसे adder). Sequential circuit = output अभी के inputs और पुरानी

memory पर निर्भर (जैसे flip-flop, counter).

- Digital उदाहरण: computer, mobile, digital camera, calculator, digital watch. (TRE (Teacher Recruitment Examination) 3.0 में "More than one" वाला सवाल.)

④ 4. Exam Point + Trap

- Exam Point: 4 inputs की truth table में 16 rows (TRE 3.0). Trap: 4 inputs देखकर "4" चुन लेना.
- Exam Point: 2 variables के कुल functions 16 (TRE 2.0). Trap: 2^2 = 4 पर रुक जाना, वह rows हैं, functions नहीं.
- Exam Point: Digital circuits तेज़ होते हैं और दो levels पर चलते हैं. "analog / continuous voltage / slow" वाले statements गलत हैं (TRE 3.0 का जवाब "None of the above").

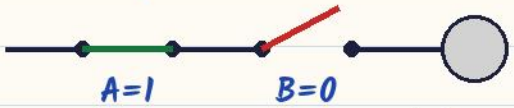
⑤ 5. याद रखो

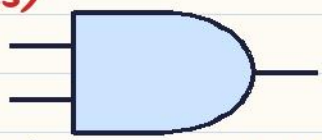
- Digital = 0/1. Boolean algebra = 0/1 की algebra. Rows = 2^n, functions = 2^(2^n). Combinational में memory नहीं, sequential में memory होती है.

1. Gates को Switch की तरह समझो

Bulb = Output (1 = जला, 0 = बुझा) · Switch = Input (1 = ON, 0 = OFF)

AND = दो switch एक line में (Series)





एक भी OFF → bulb बुझा। दोनों ON हों तभी जलेगा। ↑ AND symbol (D shape)

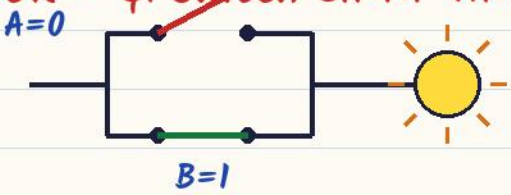
Trick: AND = 'गुणा' (×)

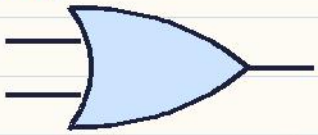
0 × कुछ भी = 0

सिर्फ 1 × 1 = 1

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR = दो switch आमने-सामने (Parallel)





↑ OR symbol (ढाल shape)

कोई भी एक ON → bulb जला।

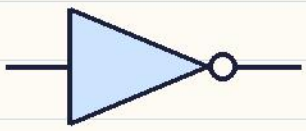
Trick: OR = 'जोड़' (+)

पर 1 + 1 = 1 (2 नहीं!)

सिर्फ 0 + 0 = 0

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT = उल्टा कर दो (Inverter)



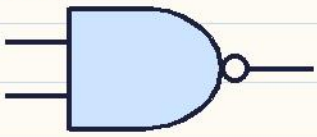
छोटा गोला (bubble) = NOT का निशान

A	Y
0	1
1	0

याद रखो: bubble जहाँ दिखे, वहाँ output उल्टा।
AND + bubble = NAND, OR + bubble = NOR

2. बाकी 4 Gates: एक नज़र में

NAND

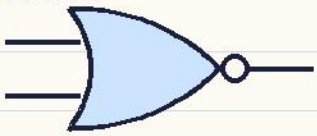


$Y = (A \cdot B)'$

AND का उल्टा: सिर्फ 1,1 पर 0

AB: `00 → 1` `01 → 1` `10 → 1` `11 → 0`

NOR

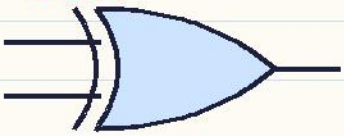


$Y = (A + B)'$

OR का उल्टा: सिर्फ 0,0 पर 1

AB: `00 → 1` `01 → 0` `10 → 0` `11 → 0`

XOR

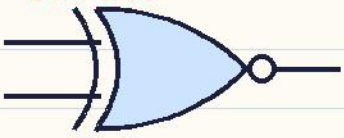


$Y = A \oplus B = A'B + AB'$

अलग-अलग input = 1 ('झगड़ा' gate)

AB: `00 → 0` `01 → 1` `10 → 1` `11 → 0`

XNOR



$Y = (A \oplus B)' = AB + A'B'$

एक जैसे input = 1 ('दोस्ती' gate)

AB: `00 → 1` `01 → 0` `10 → 0` `11 → 1`

Exam Trick (बहुत पूछा जाता है):

XOR = odd number of 1s → 1 (Parity checker में use)

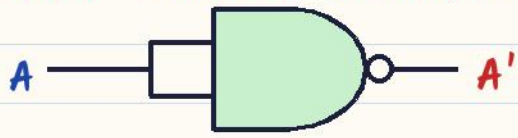
XOR = Half Adder का SUM, AND = उसका CARRY

$A \oplus 0 = A$, $A \oplus 1 = A'$, $A \oplus A = 0$, $A \oplus A' = 1$

3. Universal Gate: NAND से सब कुछ

NAND और NOR अकेले ही हर gate बना सकते हैं, इसलिए 'Universal' कहलाते हैं।

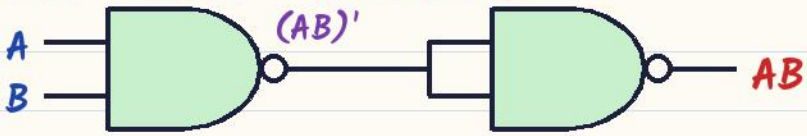
(a) NOT = NAND के दोनों input जोड़ दो



$(A \cdot A)' = A'$ ✓

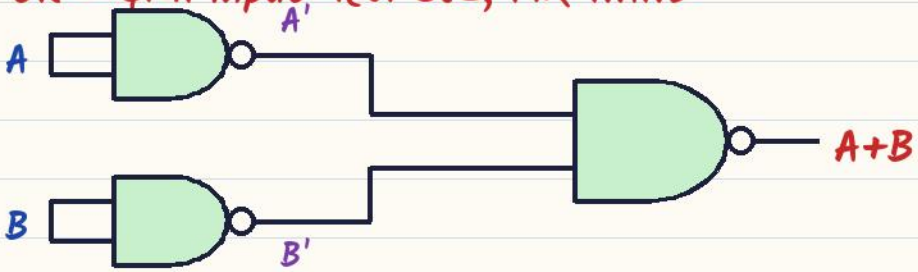
1 NAND लगता है

(b) AND = NAND + NAND(NOT)



2 NAND लगते हैं

(c) OR = दोनों input पहले उल्टे, फिर NAND



$(A' \cdot B')' = A + B$ (De Morgan) · 3 NAND लगते हैं

Exam में पूछा जाता है: कितने gate लगेंगे?

Gate	NAND से	NOR से
NOT	1	1
AND	2	3
OR	3	2
XOR	4	5
XNOR	5	4

Trick: NAND और NOR में AND ↔ OR की गिनती उलट जाती है
NAND: AND 2, OR 3 · NOR: AND 3, OR 2

Logic Gates

1. आसान भाषा में

- Logic gate एक छोटा circuit है जो एक या ज़्यादा inputs (0/1) लेकर एक output (0/1) देता है. जैसे दरवाज़े का गार्ड, जो शर्त पूरी होने पर ही गेट खोलता है.
- तीन बुनियादी gates हैं: AND, OR, NOT. इनसे बने चार और: NAND (NOT AND), NOR (NOT OR), XOR (Exclusive OR), XNOR (Exclusive NOR).

2. हर gate कैसे काम करता है

- AND: output 1 तभी जब सारे inputs 1 हों. Symbol A.B. जैसे परीक्षा पास होने के लिए लिखित और प्रैक्टिकल दोनों पास चाहिए.
- OR: output 1 अगर कोई भी एक input 1 हो. Symbol A+B. जैसे घर में आने के लिए आगे या पीछे का कोई भी दरवाज़ा.
- NOT (inverter): उल्टा कर देता है. 0 -> 1, 1 -> 0. Symbol A'.
- NAND (NOT AND): AND का उल्टा. Output 0 सिर्फ तब जब सारे inputs 1 हों. (A.B)'.
- NOR (NOT OR): OR का उल्टा. Output 1 सिर्फ तब जब सारे inputs 0 हों. (A+B)'.
- XOR (Exclusive OR): output 1 जब दोनों inputs अलग हों. A xor B = A'B + AB'. Symbol A ⊕ B.
- XNOR (Exclusive NOR): XOR का उल्टा. Output 1 जब दोनों inputs एक जैसे हों. A'B' + AB.

3. सारी truth table एक नज़र में (A, B -> AND, OR, NAND, NOR, XOR, XNOR)

- 0 0 -> 0, 0, 1, 1, 0, 1
- 0 1 -> 0, 1, 1, 0, 1, 0
- 1 0 -> 0, 1, 1, 0, 1, 0

- 1 1 -> 1, 1, 0, 0, 0, 1
- पक्का याद करने की trick: AND सिर्फ (1,1) पर 1. OR सिर्फ (0,0) पर 0. NAND सिर्फ (1,1) पर 0. NOR सिर्फ (0,0) पर 1. XOR तब 1 जब inputs अलग.

④ 4. Universal gates (बहुत पूछा जाता है)

- जिस gate से बाकी सभी gates बन जाएँ, उसे Universal gate कहते हैं. ये दो हैं: NAND और NOR. AND, OR, XOR universal नहीं हैं.
- NAND से NOT: दोनों input जोड़ दो, NAND(A,A) = A'.
- NAND से AND: NAND के बाद NOT: ((A.B)')' = A.B.
- NAND से OR: पहले दोनों को NOT, फिर NAND: (A'.B')' = A+B (De Morgan).
- NOR से भी इसी तरह: NOR(A,A) = A'. इसलिए NOR भी universal.
- TRE (Teacher Recruitment Examination) 2.0/3.0: "universal gate कौन?" -> NAND. "Digital circuit किसके बार-बार उपयोग से बनता है?" -> NAND और NOR दोनों, इसलिए "More than one of the above".

⑤ 5. Associative क्या है, और कौन नहीं है

- Associative: तीन चीज़ें हों तो (A op B) op C = A op (B op C). AND, OR, XOR associative हैं.
- NAND और NOR associative नहीं हैं. Test: A=0, B=0, C=1: (A NAND B) NAND C = 1 NAND 1 = 0, पर A NAND (B NAND C) = 0 NAND 1 = 1. दोनों अलग. (NET (National Eligibility Test) Dec 2004).

⑥ 6. XOR के काम की बातें

- A xor A = 0, A xor 0 = A, A xor 1 = A'. XOR "अलग है?" बताता है और half adder का sum देता है.
- XOR swap trick: a = a^b; b = a^b; a = b^a; से बिना तीसरे variable के a, b बदल जाते हैं (NET Dec 2005).

- Odd number of 1s हों तो multi-input XOR = 1 (parity में काम आता है).

⑦ 7. Half adder (पहला adder)

- Half adder दो 1-bit संख्याएँ जोड़ता है. Outputs दो: Sum और Carry. Sum = A xor B, Carry = A.B.
- 0+0 = 00, 0+1 = 01, 1+0 = 01, 1+1 = 10 (sum 0, carry 1). इसीलिए NET में इसे "EX-OR circuit" कहा गया (sum की वजह से), पर असल में XOR + AND दोनों लगते हैं.
- Half subtractor के outputs: Difference और Borrow. TRE 3.0: "half adder के outputs?" -> Sum और Carry.

⑧ 8. C++ वाला नज़रिया

- C++ में bit-wise operators: & (AND), | (OR), ^ (XOR), ~ (NOT, हर bit पलटो). Logical operators: && , || , ! (सिर्फ true/false).
- TRE 3.0: 1010 AND 1100 = 1000. यानी 10 & 12 = 8. बिट-दर-बिट AND करो.

```
int a = 10, b = 12;      // 1010 □□ 1100
cout << (a & b);         // 8  (1000)
cout << (a | b);         // 14 (1110)
cout << (a ^ b);         // 6  (0110)
```

⑨ 9. Exam Point + Trap

- Exam Point: Half adder = XOR (sum) + AND (carry). Trap: "full adder" में तीन inputs होते हैं (A, B, और carry-in), half adder में दो.
- Exam Point: Universal = NAND और NOR. Trap: XOR को universal मान लेना.
- Exam Point: NAND और NOR associative नहीं हैं. Trap: "XOR भी nonassociative" सोचना, यह गलत है.

⑩ 10. याद रखो

- AND = सब 1, OR = कोई एक 1, XOR = अलग-अलग. Universal =

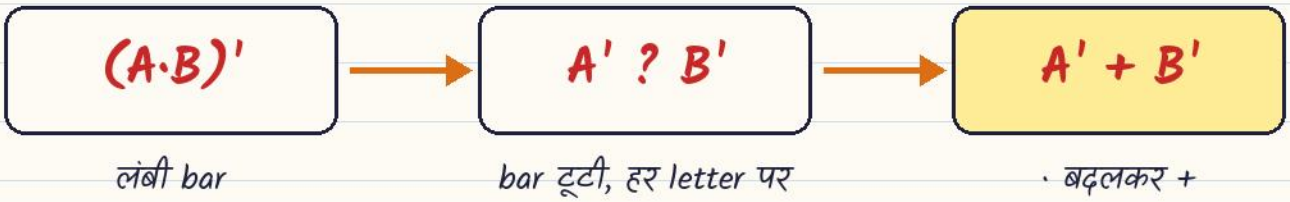
NAND, NOR. Half adder: sum = XOR, carry = AND.

4. De Morgan: 'Bar तोड़ो, Sign बदलो'

$$(A \cdot B)' = A' + B'$$

$$(A + B)' = A' \cdot B'$$

Step-by-step (तस्वीर से):



Gate में मतलब: NAND = bubble वाले inputs का OR



जरूरी Laws (रटो नहीं, 1/0 डालकर check करो):

$A + 0 = A$	$A \cdot 1 = A$	Identity
$A + 1 = 1$	$A \cdot 0 = 0$	Null/Dominance
$A + A = A$	$A \cdot A = A$	Idempotent
$A + A' = 1$	$A \cdot A' = 0$	Complement
$A + AB = A$	$A(A + B) = A$	Absorption
$A + A'B = A + B$	$A(A' + B) = AB$	Redundancy

Absorption trick: $A + AB$ में A पहले से है, तो AB 'खा' लिया जाता है → A
Duality: $+ \leftrightarrow \cdot$ और $0 \leftrightarrow 1$ बदलो, दूसरा law मिल जाता है

Boolean Laws

1 I. आसान भाषा में

- Boolean laws वे नियम हैं जिनसे लंबे expression को छोटा किया जाता है. छोटा expression = कम gates = सस्ता और तेज़ circuit.
- हर नियम की जाँच दो तरह से हो सकती है: (1) truth table बनाकर, (2) A=0 और A=1 रखकर देख लो. Exam में तेज़ तरीका यही दूसरा है.

2 2. मुख्य नियम (AND/OR जोड़ी में)

- Identity: A + 0 = A और A . 1 = A. (0 जोड़ने या 1 से गुणा करने पर कुछ नहीं बदलता.)
- Null / Domination: A + 1 = 1 और A . 0 = 0.
- Idempotent: A + A = A और A . A = A. (अपने साथ करने पर वही.) NET (National Eligibility Test) Dec 2004 में पूछा गया.
- Complement: A + A' = 1 और A . A' = 0.
- Involution / Double negation: (A')' = A. (दो बार उल्टा = सीधा). TRE (Teacher Recruitment Examination) 2.0: (((x')')')' = x.
- Commutative: A + B = B + A, A.B = B.A. Associative: (A+B)+C = A+(B+C).
- Distributive: A(B + C) = AB + AC. और खास बात: A + BC = (A+B)(A+C) (साधारण algebra में ऐसा नहीं होता). TRE 2.0 में पूछा गया.
- Absorption: A + AB = A और A(A + B) = A. TRE 2.0 में x + xy = x और x(x + y) = x.
- Absorption का दूसरा रूप: A + A'B = A + B. और A(A' + B) = AB.
- Consensus: AB + A'C + BC = AB + A'C (बीच वाला BC बेकार). NET Dec 2007: xy + x'z + yz = xy + x'z.

3 3. De Morgan के नियम (सबसे ज़रूरी)

- (A . B)' = A' + B' यानी AND का NOT = अलग-अलग NOT का OR.

- (A + B)' = A' . B' यानी OR का NOT = अलग-अलग NOT का AND.
- आसान याद: "बार (bar) तोड़ो, चिन्ह बदलो". NOT के नीचे का AND OR बन जाता है, OR AND बन जाता है.
- फायदा: AND/OR circuit को NAND (NOT AND) या NOR (NOT OR) gates से बनाना. TRE 3.0 में पूछा: De Morgan किसके काम आता है? -> Expression simplify करना और NAND gates से implement करना, दोनों (More than one).

④ 4. Duality (द्वैत)

- Dual लेने का तरीका: AND <-> OR बदलो, 0 <-> 1 बदलो, variables जैसे के तैसे. जैसे A + 0 = A का dual A . 1 = A. हर Boolean नियम का dual भी सही होता है.

⑤ 5. Exam Point + Trap

- Exam Point: A + BC = (A+B)(A+C) पक्का याद करो. Trap: (A+B)(A+C) को A + B + C मत समझो.
- Exam Point: x + xy = x (absorption) बनाम x + x'y = x + y. Trap: दोनों उलझा देते हैं. देखो: जब दूसरे term में x की जगह x' हो तब x + y.
- Exam Point: (((x')')')' = x. गिनती: NOT सम तो x, विषम तो x'.

⑥ 6. याद रखो

- Absorption: A + AB = A. Complement: A + A' = 1. De Morgan: बार तोड़ो, चिन्ह बदलो. Distributive: A + BC = (A+B)(A+C).

Simplification Steps

① 1. आसान भाषा में

- Simplification यानी लंबे Boolean expression को छोटा करना. बिना किसी नतीजे के बदलाव के. तरीका: common variable निकालो, नियम लगाओ, और अंत में test करो.

② 2. कदम-दर-कदम तरीका

- Step 1: दो terms में कोई variable common हो तो bracket में निकालो. जैसे AB + AB' = A(B + B') = A.
- Step 2: bracket के अंदर X + X' = 1 या X.X' = 0 जैसी चीज़ें खोजो.
- Step 3: absorption (A + AB = A) लगाओ.
- Step 4: ज़रूरत हो तो consensus (बेकार term हटाओ).
- Step 5: Test: 1-2 values (जैसे सब 0, सब 1) रखकर मूल और नतीजा मिलाओ. यह MCQ में तेज़ तरीका है.

③ 3. हल किये हुए उदाहरण (सारे असली PYQ)

- TRE (Teacher Recruitment Examination) 2.0: AB + AB' + A'C + AC कौन से variable से independent? -> AB + AB' = A; A'C + AC = C; F = A + C. इसमें B नहीं, इसलिए B से independent.
- TRE 2.0: F = A'C + A'B + AB'C + BC = ? -> A'C + AB'C = C(A' + B'); B'C + BC = C; बचा C + A'B. जवाब C + A'B.
- NET (National Eligibility Test) Dec 2007: xy + x'z + yz = xy + x'z (consensus).
- NET Dec 2009: (X + Y + XY)(X + Z). पहले X + Y + XY = X + Y, फिर (X + Y)(X + Z) = X + YZ.
- NET Jun 2009: x'y'z + yz + xz. z निकालो: z(x'y' + y + x). x'y' = (x+y)' (De Morgan), इसलिए अंदर (x+y)' + (x+y) = 1. जवाब z.

④ 4. Exam Point + Trap

- Exam Point: x' y' को (x + y)' पहचानना सीखो (De Morgan). यही NET के कई सवालों की चाबी है.
- Trap: simplify करने के बाद options मिलाते समय option का रूप अलग हो सकता है (जैसे YZ + X की जगह X + YZ). दोनों बराबर हैं.
- Trap: मुश्किल लगे तो सीधे values रखो: सब variables = 0, फिर सब = 1, और मूल expression से option मिलाओ.

⑤ 5. याद रखो

- Common निकालो, X + X' = 1 ढूँढो, absorption लगाओ, फिर test करो.

5. SOP और POS: Truth Table से Equation

Example: Y = 1 जब A, B, C में से कम से कम दो 1 हों (Majority)

m	A	B	C	Y
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

SOP (पीले rows, Y = 1 वाले)

हर 1 वाली row का AND term:

0 → A' (bar लगाओ), 1 → A

m3 = 011 → A'BC

m5 = 101 → AB'C

m6 = 110 → ABC'

m7 = 111 → ABC

$Y = A'BC + AB'C + ABC' + ABC$

$= \Sigma m(3, 5, 6, 7)$

POS (नीले rows, Y = 0 वाले)

हर 0 वाली row का OR term:

0 → A, 1 → A' (उल्टा!)

$Y = \Pi M(0, 1, 2, 4)$

Shortcut याद रखो:

SOP = Sum of minterms = 1 वाले rows · **Σ** लिखो

POS = Product of Maxterms = 0 वाले rows · **Π** लिखो

Σ और **Π** के numbers मिलाकर हमेशा 0 से 2^n-1 तक पूरे होते हैं

$\Sigma m(3,5,6,7) = \Pi M(0,1,2,4)$ ✓

Exam point:

n variable → कुल 2^n minterms · 3 variable → 8 rows

Minterm में हर variable आता है (Canonical form)

Simplify करके: $Y = AB + BC + AC$ (Majority function)

SOP, POS, Canonical

1. I. आसान भाषा में

- Boolean function को लिखने के दो मानक तरीके हैं. SOP (Sum of Products) (Sum of Products): पहले AND (products), फिर उन्हें OR से जोड़ो. जैसे F = AB + CD.
- POS (Product of Sums) (Product of Sums): पहले OR (sums), फिर उनका AND. जैसे F = (A+B)(C+D).

2. 2. Circuit से रिश्ता

- SOP का circuit: पहले AND gates, फिर एक OR gate (AND-OR). TRE (Teacher Recruitment Examination)/NET (National Eligibility Test) में पूछा: "SOP को किससे बनाते हैं?" -> AND-OR.
- POS का circuit: पहले OR gates, फिर AND (OR-AND).
- NAND-NAND (NOT AND) भी SOP देता है: सब AND-OR की जगह सिर्फ NAND लगाओ (De Morgan की वजह से). इसलिए आसान.

3. 3. Canonical forms: Minterm और Maxterm

- Canonical form में हर term में सारे variables मौजूद होते हैं. इसके दो रूप: SOM (Sum of Minterms) (Sum of Minterms) और POM (Product of Maxterms) (Product of Maxterms). (TRE 1.0 में पूछा.)
- Minterm = सारे variables का AND, जहाँ value 1 वाला variable सीधा और 0 वाला उल्टा (NOT). 2 variables: m0 = A'B', m1 = A'B, m2 = AB', m3 = AB.
- Maxterm = सारे variables का OR, जहाँ 0 वाला सीधा और 1 वाला उल्टा. M0 = A+B, M1 = A+B', M2 = A'+B, M3 = A'+B'.
- SOM में सिर्फ वे minterms लो जहाँ function का output 1 है. POM में वे maxterms जहाँ output 0 है.
- n variables के कुल minterms (और maxterms) = 2^n. 3 variables -> 8.

④ 4. Example: SOM निकालना

- F(A,B) की table: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0. यह XOR (Exclusive OR) है.
- Output 1 वाली rows: (0,1) और (1,0). SOM = A'B + AB' = m1 + m2 = Σm(1,2).
- Output 0 वाली rows: (0,0) और (1,1). POM = (A+B)(A'+B') = M0 . M3 = ΠM(0,3).
- दोनों रूप एक ही function हैं. Σ का मतलब sum (OR), Π का मतलब product (AND).

⑤ 5. NAND से XOR (4 gates)

- TRE 2.0: x'y + xy' के लिए कम से कम कितने NAND gates? -> 4.
- N1 = NAND(x,y); N2 = NAND(x,N1) = x' + y; N3 = NAND(y,N1) = y' + x; Output = NAND(N2,N3) = XOR.
- पक्की बात: एक XOR = 4 NAND gates. एक XOR = 5 NOR (NOT OR) gates भी बनता है (NOR से 5).

⑥ 6. Exam Point + Trap

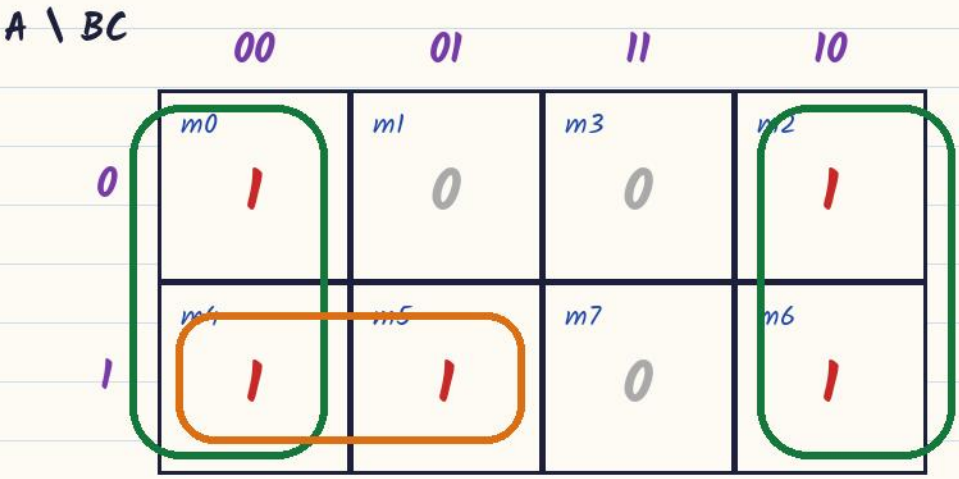
- Exam Point (TRE 1.0): Y = AB + BC + AC -> SOP. Trap: "यह तो XOR है" सोचना; चिन्ह देखो, products का + से जुड़ना SOP है.
- Exam Point: Canonical forms = SOM और POM. Trap: OR/XOR gates के नाम नहीं हैं canonical form.
- Trap: Minterm में 1 वाला variable सीधा, Maxterm में 1 वाला उल्टा. उल्टा याद रखने से गलती होती है.

⑦ 7. याद रखो

- SOP = AND फिर OR. POS = OR फिर AND. Canonical = हर term में सारे variables: SOM, POM. XOR = 4 NAND.

6. K-Map: तस्वीर से Simplify करो

F(A,B,C) = Σm(0, 2, 4, 5, 6) को simplify करना है



हरा group (4 cells): किनारे जुड़ते हैं! C = 0 हर जगह → C'

नारंगी group (2): A=1, B=0 → AB'

$$F = C' + AB'$$

K-map के 5 Golden Rules:

1. Columns का order 00, 01, 11, 10 (Gray code): पड़ोसी में सिर्फ 1 bit बदले
2. Group का size सिर्फ 1, 2, 4, 8... (2 की power)
3. Group जितना बड़ा, term उतना छोटा · कम से कम groups बनाओ
4. बाएँ-दाएँ और ऊपर-नीचे के किनारे जुड़े हैं (wrap around)
5. Group में जो variable बदलता है, उसे हटा दो; जो fixed है वही लिखो

Quick count: n variable K-map में
2 cells group → 1 variable हटेगा · 4 cells → 2 हटेंगे · 8 cells → 3 हटेंगे
Don't care (X) को 1 मानो अगर group बड़ा बने, वरना छोड़ दो

K-map & Don't care

1. आसान भाषा में

- K-map (Karnaugh map) (Karnaugh map) एक चित्र वाली table है जिसमें Boolean function के 1 को पास-पास बैठाकर समूह (group) बनाते हैं. समूह बनाने से algebra अपने आप सरल हो जाता है.
- n variables के लिए 2^n cells (खाने). 2 variables = 4 cells, 3 = 8, 4 = 16, 5 = 32, 6 = 64. NET (National Eligibility Test) Jun 2006: K-map अधिकतम 6 variables तक.

2. Gray order क्यों

- K-map की rows/columns का क्रम 00, 01, 11, 10 होता है (Gray code), 00, 01, 10, 11 नहीं. क्योंकि पास वाले cells में सिर्फ एक variable बदलना चाहिए, तभी वे जुड़ सकते हैं.

3. Grouping के नियम

- सिर्फ 1 को group करो (SOP (Sum of Products) के लिए). Group का आकार 1, 2, 4, 8, 16 (2 की power) ही हो सकता है.
- Group जितना बड़ा, उतना सरल term. किनारे घूमकर भी जुड़ सकते हैं (wrap-around): ऊपर-नीचे और बाएँ-दाएँ.
- तिरछे (diagonal) cells group नहीं होते. Overlap (एक cell दो group में) चलता है.
- Group में जो variable बदलता है वह term से हट जाता है.

4. Example (2 variable)

- F = A'B + AB + A'B' के 1 वाले cells: A'B', A'B, AB. A=0 की पूरी row (A'B' और A'B) एक group -> A'. B=1 वाला column (A'B और AB) एक group -> B.
- Result F = A' + B. Check: A=1, B=0 पर F = 0 है, सही (क्योंकि A'B' और A'B और AB में AB' नहीं).

⑤ 5. Don't care (X)

- Don't care वह input combination है जो कभी आती ही नहीं या जिसके output की हमें परवाह नहीं. इसे X लिखते हैं.
- ज़रूरत हो तो X को 1 मान लो (ताकि बड़ा group बने), नहीं तो 0. यह छूट circuit को छोटा करती है. (इसका TRE (Teacher Recruitment Examination) 3.0 सवाल Part 2 में आएगा, जहाँ K-map विस्तार से पढ़ोगे.)

⑥ 6. Exam Point + Trap

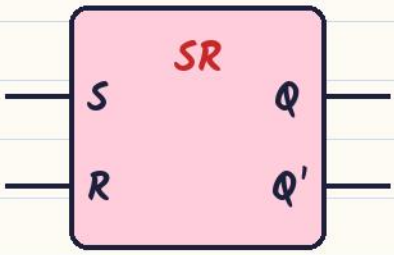
- Exam Point: K-map 6 variable तक. Trap: 4 variable समझना (वह सबसे आम है, limit नहीं).
- Trap: Gray order 00,01,11,10 है, साधारण binary order नहीं.

⑦ 7. याद रखो

- 1 को 2 की power वाले groups में बाँधो, बड़ा group बेहतर. X चाहे तो 1 मानो. K-map 6 variable तक.

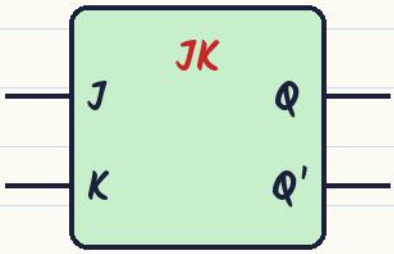
7. Flip-Flop: 1 bit याद रखने वाला डिब्बा

Gate = याददाश्त नहीं (Combinational) · Flip-flop = पिछला output याद (Sequential)



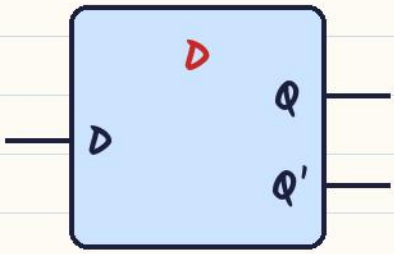
S = Set (1 करो), R = Reset (0 करो)

- 00 → Q(next) = Q (no change)
- 01 → Q(next) = 0 (Reset)
- 10 → Q(next) = 1 (Set)
- 11 → Q(next) = Invalid ✘



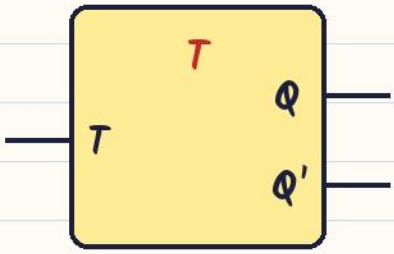
SR जैसा, पर 11 पर Toggle · invalid हटाया

- 00 → Q(next) = Q (no change)
- 01 → Q(next) = 0 (Reset)
- 10 → Q(next) = 1 (Set)
- 11 → Q(next) = Q' (Toggle) ✔



D = Data/Delay: जो D वही Q (next clock पर)

- 0 → Q(next) = 0
- 1 → Q(next) = 1



T = Toggle: T=1 पर पलट जाता है

- 0 → Q(next) = Q (no change)
- 1 → Q(next) = Q' (Toggle)

Trick: 'SR में 11 मना, JK में 11 पलटा'
JK के J और K जोड़ दो → T flip-flop · SR में R = S' → D flip-flop
JK की Race-around problem → Master-Slave JK से हल

Flip-Flops

1. आसान भाषा में

- अब तक के gates में memory नहीं थी: input हटा तो output बदल गया. Flip-flop एक ऐसा circuit है जो 1 bit याद रखता है. इसी से register, counter और memory बनते हैं.
- Flip-flop के दो stable states हैं (0 और 1). Output को Q लिखते हैं. Q(n) = अभी की value (present state), Q(n+1) = clock के बाद की value (next state).

2. रोज़मर्रा का example

- दीवार का वह switch जो दबाने पर ON रहता है और अगली बार दबाने पर OFF. स्विच अपनी स्थिति याद रखता है. Flip-flop भी अपनी स्थिति याद रखता है.

3. Clock और Triggering (TRE 3.0)

- Clock हर थोड़े समय पर 0-1-0-1 की पल्स भेजता है. Flip-flop इसी के इशारे (trigger) पर बदलता है.
- Level triggered: clock जिस level (0 या 1) पर है उतनी देर flip-flop बदल सकता है. Positive level triggered = clock 1 पर बदलेगा. Negative level triggered = clock 0 (negative/low) पर बदलेगा.
- Edge triggered: सिर्फ clock के बदलने के क्षण पर. Positive edge = 0 से 1 होते समय. Negative edge = 1 से 0 होते समय.
- TRE (Teacher Recruitment Examination) 3.0: "negative level triggered flip-flop कब बदलता है?" -> जब clock negative हो.

4. चार flip-flops और उनके equations (NET में बार-बार)

- SR (Set-Reset): S=1 -> Q=1 (set), R=1 -> Q=0 (reset), S=R=0 -> वही रहे (hold), S=R=1 मना है (invalid).
- D (Data/Delay): Q(n+1) = D. जो D में दोगे वही अगली clock पर Q बन जाएगा. S और R के बीच एक inverter लगाने से SR से D बनता है.

- JK: SR का सुधरा रूप. J=K=0 hold, J=0,K=1 reset, J=1,K=0 set, J=K=1 toggle. Equation: Q(n+1) = J.Q' + K'.Q (NET (National Eligibility Test) Jun 2009).
- T (Toggle): T=0 hold, T=1 पलटो. Equation: Q(n+1) = T xor Q = T'Q + TQ'. (NET Dec 2004 और Jun 2008).
- JK के J और K को जोड़कर T बनता है (J=K=T). J और K के बीच inverter लगाओ (J=D, K=D') तो D flip-flop बनता है.

5. 5. Truth table (JK और T)

- JK: (J,K) = (0,0) -> Q; (0,1) -> 0; (1,0) -> 1; (1,1) -> Q'.
- T: T=0 -> Q; T=1 -> Q'.
- जाँच: JK के equation Q+ = JQ' + K'Q में J=1,K=0 रखो -> Q' + Q = 1 (set). J=K=1 -> Q' + 0 = Q' (toggle).
- NET Dec 2005 में पूछा गया "S और R के बीच inverter लगाने पर क्या बनेगा?" - key में अस्पष्ट सवाल मानकर "None of these" है, पर किताबों में यह D flip-flop होता है.

6. 6. Exam Point + Trap

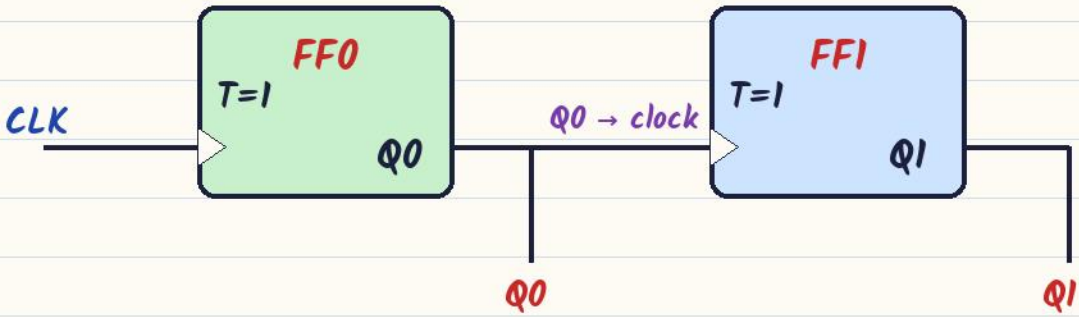
- Exam Point: T = XOR (Exclusive OR), D = Q+ = D, JK = JQ' + K'Q. Trap: T का equation AND (T.Q) या OR (T+Q) समझ लेना.
- Exam Point: SR में S=R=1 invalid है, JK में यही toggle बन जाता है. यह JK का फायदा.
- Trap: Level triggered और Edge triggered को मिला देना.

7. 7. याद रखो

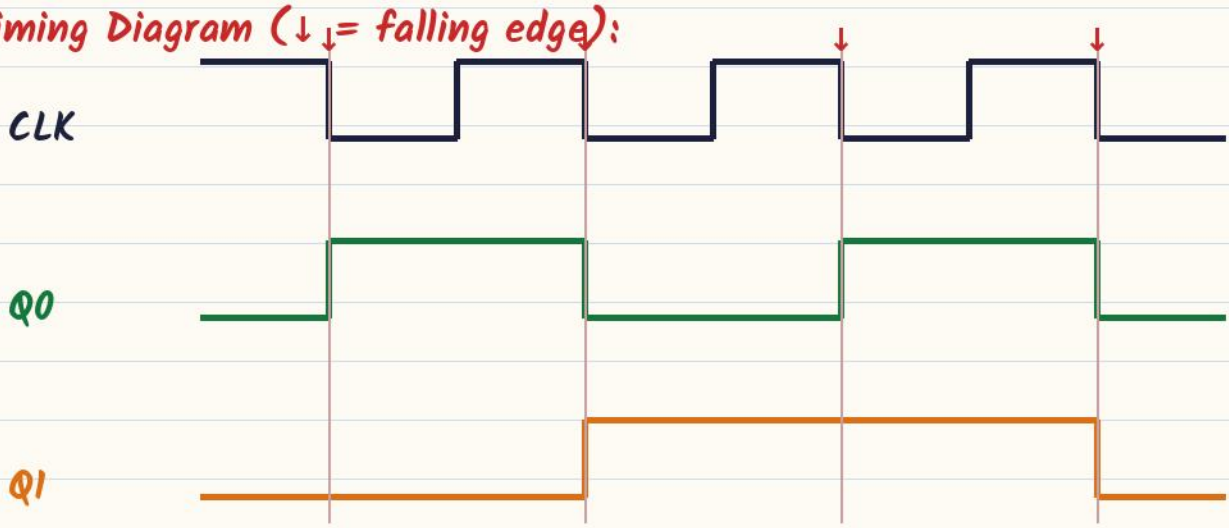
- T: Q+ = T xor Q. JK: Q+ = JQ' + K'Q. D: Q+ = D. Negative level triggered = clock negative (0) पर बदलता है.

8. Counter: Flip-Flop से गिनती

2-bit Ripple (Asynchronous) Counter: दो T flip-flop (T = 1), नीचे जाते clock edge पर



Timing Diagram (↓ = falling edge):



Q1 Q0 क्रम से: 00 → 01 → 10 → 11 → 00 ... (0,1,2,3 गिनती)

Formula (exam favourite):

n flip-flop → 2^n states · MOD-2^n counter · max count = $2^n - 1$

हर flip-flop frequency को आधा करता है: n FF → $f / 2^n$

MOD-N के लिए कम से कम n FF, जहाँ $2^n \geq N$ (जैसे MOD-10 → 4 FF)

Ripple: clock एक-एक करके (slow, delay = n × tpd)

Synchronous: सबको एक साथ clock (fast)

Counters

1. I. आसान भाषा में

- Counter flip-flops का वह group है जो clock की पल्स गिनता है. जैसे cycle का meter जो हर चक्कर पर एक बढ़ता है.
- Ripple (asynchronous) counter: पहले flip-flop को clock मिलता है, बाकी को अपने से पहले वाले का output. इसलिए बदलाव लहर की तरह (ripple) आगे फैलता है.

2. 2. Counter की गिनती के नियम

- n flip-flops से 2^n अलग states (0 से 2^n - 1) बनती हैं. 4 flip-flop -> 16 states (0 से 15). 5 -> 32 states.
- MOD-N counter = N states तक गिनता है (0 से N-1) और फिर 0 पर लौटता है. न्यूनतम flip-flops n: 2^n >= N यानी n = ceil(log2 N).
- MOD-18 के लिए: 2^4 = 16 < 18, 2^5 = 32 >= 18. इसलिए 5 flip-flops. (NET (National Eligibility Test) Dec 2007).
- Binary counter के MOD-16 (4-bit) में 16 तक गिनती. 4-bit counter पूरा चक्र = 16 पल्स.

3. 3. Maximum counting speed (NET Jun 2007)

- Ripple counter में हर flip-flop की delay (tpd (propagation delay) = propagation delay) जुड़ती है. Total delay = n x tpd.
- Maximum frequency = 1 / (n x tpd).
- 4-bit counter, tpd = 25 ns: 4 x 25 = 100 ns. f = 1 / 100 ns = 10 MHz (Mega Hertz). (जवाब 10 MHz).
- Trap: 1 / 25 ns = 40 MHz लेना गलत है, क्योंकि सारी delays जुड़ती हैं.

4. 4. Exam Point + Trap

- Trap: MOD-N में flip-flops की संख्या N नहीं है, log2 N ऊपर गोल करके.

- Trap: ns को MHz में बदलना: 1 ns = 10^-9 s; 100 ns -> 10^7 Hz = 10 MHz.

⑤ 5. याद रखो

- n flip-flop = 2^n states. MOD-N के लिए 2^n >= N. f max = 1 / (n x tpd).

Logic Families

① 1. आसान भाषा में

- Logic family यानी gates बनाने की तकनीक (technology). एक ही AND gate अलग-अलग तकनीक से बन सकता है. हर family की अपनी गति, power, और शोर सहने की क्षमता होती है.
- नाम: RTL (Resistor Transistor Logic), DTL (Diode Transistor Logic), TTL (Transistor-Transistor Logic), ECL (Emitter Coupled Logic), MOS (Metal Oxide Semiconductor), CMOS (Complementary MOS (Metal Oxide Semiconductor)), BiCMOS (Bipolar + CMOS (Complementary Metal Oxide Semiconductor)). RTL (Resistor Transistor Logic)/DTL (Diode Transistor Logic) पुरानी हैं.

② 2. जरूरी शब्द

- Propagation delay: input बदलने से output बदलने तक का समय. कम delay = तेज़.
- Power dissipation: gate कितनी बिजली खाता है.
- Fan out: एक gate का output ज़्यादा से ज़्यादा कितने gates के input को चला सकता है.
- Noise margin: कितना बाहरी शोर सहने पर भी 0 और 1 सही पहचाने जाएँ.

③ 3. कौन-सी family कैसी (NET के 7 सवाल यहीं से)

- सबसे तेज़: ECL (Emitter Coupled Logic). (NET (National Eligibility Test) Dec 2006, Jun 2008.) कारण: transistor saturation में नहीं जाते. कमी: power ज़्यादा.
- सबसे कम power: CMOS. (NET Dec 2007.) CMOS में कम power और कम cost per gate, इसलिए आज के chips इसी से बनते हैं (NET Dec 2008: CMOS).
- सबसे ज़्यादा fan out: CMOS. (NET Jun 2005.)
- सबसे ज़्यादा noise margin: CMOS. (NET Dec 2009; bank key TTL (Transistor-Transistor Logic) लिखी है, पर CMOS का noise margin सबसे ज़्यादा (Vdd का लगभग 45%) है.)
- CMOS का threshold/transition voltage = Vdd/2. 10 V supply पर 5 V (NET Jun 2010).
- RTL: सबसे धीमी, पुरानी. TTL: मध्यम गति (लगभग 10 ns), आम इस्तेमाल. BiCMOS (Bipolar + CMOS): bipolar की गति + CMOS की कम power.

④ 4. एक नज़र में table

- तेज़ी: ECL > TTL > RTL (RTL सबसे धीमी पुरानी). Power: CMOS सबसे कम, ECL सबसे ज़्यादा.
- Fan out और noise margin: CMOS सबसे अच्छा. Cost per gate (छोटा size): CMOS सबसे अच्छा.

⑤ 5. Exam Point + Trap

- Exam Point: Fastest = ECL; Least power = CMOS; Max fan out = CMOS; Max noise margin = CMOS; Threshold = Vdd/2.
- Trap: "सबसे तेज़" और "सबसे कम power" दोनों का उत्तर अलग है (ECL बनाम CMOS).

⑥ 6. याद रखो

- ECL तेज़, CMOS कम power + ज़्यादा fan out + अच्छा noise margin,

TTL सामान्य, RTL पुराना धीमा.

BPSC ऐसे पूछता है

① BPSC ऐसे पूछता है (TRE 1.0/2.0/3.0) — Boolean Logic

- TRE (Teacher Recruitment Examination) में हर सवाल के 5 options होते हैं: (A), (B), (C), (D) More than one of the above, (E) None of the above. आज के सवालों के type:
- Type 1 (सीधी definition/पहचान): "Y = AB + BC + AC कौन-सी operation?" -> SOP (TRE 1.0). "Canonical forms?" -> SOM (Sum of Minterms) और POM (Product of Maxterms). "PLA (Programmable Logic Array)?" (Part 2). तरीका: चिन्ह देखकर नाम पहचानो.
- Type 2 (Boolean नियम): x + x'y, A + BC, x + xy, x(x + y), (((x')')')'. तरीका: absorption/distributive याद रखो या values रखकर जाँचो. ये TRE 2.0 में 6 सवाल थे.
- Type 3 (गिनती वाले): "4 inputs की table में rows?" -> 16. "2 variables के functions?" -> 16. "XOR (Exclusive OR) के लिए NAND (NOT AND) gates?" -> 4. तरीका: formulas 2^n और 2^(2^n).
- Type 4 (Simplify): "F = A'C + A'B + AB'C + BC का सरल रूप?" -> C + A'B. तरीका: common निकालो, B + B' = 1.
- Type 5 ("More than one" या "None" वाले): "Universal gate?" NAND अकेला सही, इसलिए (C). पर "digital circuit किसके repeated use से?" NAND और NOR (NOT OR) दोनों -> (D). "Digital circuits के बारे में सही?" तीनों गलत -> (E). हर option खुद जाँचो.
- Type 6 (Bit-wise): 1010 AND 1100 = 1000.
- Type 7 (Flip-flop/Adder): "Half adder के outputs?" -> Sum और Carry. "Negative level triggered?" -> clock negative पर.

② Exam Point + Trap

- Trap 1: "More than one of the above" तब चुनो जब दो या ज़्यादा options सही हों. एक ही सही हो तो वही चुनो.
- Trap 2: Statement वाले सवालों में हर option अलग से जाँचो.
- Trap 3: x + xy = x और x + x'y = x + y में फर्क.

NET level में ऐसे आता है

① NET level में ऐसे आता है — Boolean Logic

- Type 1 (नियम का नाम): "A + A = A" -> Idempotent law (NET (National Eligibility Test) Dec 2004). Mini: A.A = A भी Idempotent.
- Type 2 (Simplify): xy + x'z + yz -> xy + x'z (consensus). (X+Y+XY)(X+Z) -> X + YZ. x'y'z + yz + xz -> z. तरीका: z जैसी common चीज़ निकालो.
- Type 3 (Gate पहचानना): "Non-associative?" NAND (NOT AND). "Universal?" NOR (NOT OR). "Half adder = ?" XOR (sum). "SOP (Sum of Products) किससे?" AND-OR.
- Type 4 (Flip-flop equation): T: Q+ = T xor Q; JK: Q+ = JQ' + K'Q. Mini: T=1,Q=1 पर Q+ = 1 xor 1 = 0.
- Type 5 (Counter गणना): MOD-18 के लिए 5 flip-flops. 4-bit counter 25 ns delay -> 10 MHz (Mega Hertz).
- Type 6 (Logic family): fastest = ECL (Emitter Coupled Logic), least power = CMOS (Complementary Metal Oxide Semiconductor), highest fan out = CMOS, CMOS threshold = Vdd/2.
- Type 7 (Program/XOR (Exclusive OR) swap): a = a^b; b = a^b; a = b^a. Mini: a=5, b=3 -> a=3, b=5. यह swap है.
- Type 8 (K-map (Karnaugh map)): K-map max 6 variables.

② Exam Point + Trap

- Trap: कभी-कभी bank की key गलत छपी होती है (जैसे T flip-flop). हमने ऐसे 4 सवालों में अपनी गणना से सही उत्तर लिखा और explanation में कारण दिया है.
- Trap: नियम याद न आए तो A, B की values 0/1 रखकर 2-3 बार test करो.

Numericals — और practice

① Practice 1: Simplify

- सवाल: F = AB + A'C + BC को सरल करो.
- Consensus: BC बेकार है. F = AB + A'C. (जाँच: A=1,B=1,C=0: मूल = 1+0+0 = 1, नतीजा = 1 सही.)

② Practice 2: Simplify

- सवाल: F = (A + B)(A + B') = ?
- A + BB' (distributive उल्टा) = A + 0 = A. जवाब A.

③ Practice 3: Simplify

- सवाल: F = A + A'B + A'B' को सरल करो.
- A + A'B = A + B. फिर A + B + A'B' = A + B + (A+B)' = 1. जवाब 1.

④ Practice 4: Truth-table rows

- सवाल: 5 inputs वाले AND gate की truth table में कितनी rows और 3-variable के कुल कितने functions?
- Rows = 2^5 = 32. 3 variables: rows = 8, functions = 2^8 = 256.

⑤ Practice 5: MOD counter

- सवाल: MOD-100 counter के लिए कम से कम कितने flip-flops?

- 2^6 = 64 < 100, 2^7 = 128 >= 100. जवाब 7.

6 Practice 6: Counter speed

- सवाल: 6-bit ripple counter, flip-flop की delay 10 ns. Max frequency?
- Total delay = 6 x 10 = 60 ns. f = 1 / 60 ns = 16.67 MHz (लगभग).

7 Practice 7: Bit-wise

- सवाल: 1101 OR 0110 और 1101 XOR (Exclusive OR) 0110?
- OR: 1,1,1,1 = 1111. XOR: 1^0=1, 1^1=0, 0^1=1, 1^0=1 -> 1011.

8 Practice 8: De Morgan

- सवाल: (A + B'C)' को सरल करो.
- De Morgan: A' . (B'C)' = A' . (B + C') = A'B + A'C'.

9 Practice 9: CMOS threshold

- सवाल: CMOS (Complementary Metal Oxide Semiconductor) supply 12 V हो तो transition voltage?
- 12 / 2 = 6 V.

Abbreviations

1 Abbreviations — Full Forms

- AND, OR, NOT = Boolean operations (AND = और, OR = या, NOT = उल्टा)
- NAND = NOT AND; NOR = NOT OR
- XOR = Exclusive OR; XNOR = Exclusive NOR
- SOP = Sum of Products; POS = Product of Sums
- SOM = Sum of Minterms; POM = Product of Maxterms
- K-map = Karnaugh map

- SR = Set-Reset; JK = (Jack Kilby के नाम पर माना जाता है); T = Toggle; D = Data/Delay (flip-flop)
- MOD = Modulus (गिनती का दायरा)
- tpd = Propagation Delay; MHz = Mega Hertz; ns = nano second (10^-9 s)
- Vdd = Drain supply voltage
- RTL = Resistor Transistor Logic; DTL = Diode Transistor Logic; TTL = Transistor-Transistor Logic
- ECL = Emitter Coupled Logic; HTL = High Threshold Logic
- MOS = Metal Oxide Semiconductor; CMOS = Complementary MOS; NMOS = N-type MOS; PMOS = P-type MOS; BiCMOS = Bipolar + CMOS
- IC = Integrated Circuit; PLA = Programmable Logic Array
- bit = Binary Digit
- NET = National Eligibility Test; UGC = University Grants Commission
- TRE = Teacher Recruitment Examination; BPSC = Bihar Public Service Commission
- PGT = Post Graduate Teacher; CS = Computer Science
- NCERT = National Council of Educational Research and Training; SCERT = State Council of Educational Research and Training; BSEB = Bihar School Examination Board

Questions + हर सवाल के नीचे Explanation

Q1. [BPSC TRE 1.0 (2023)]

The expression Y=AB+BC+AC shows the _____ operation.

(A) EX-OR
(B) SOP
(C) POS
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: EX-OR (XOR (Exclusive OR)) में A xor B जैसा चिन्ह होता है, यहाँ products का sum है
(B) ✔ सही: SOP (Sum of Products): AB, BC, AC तीन products हैं और बीच में + (OR) है
(C) गलत: POS (Product of Sums) में brackets के sums का गुणा होता है, जैसे (A+B)(B+C)
(D) गलत: सिर्फ B सही है, इसलिए "More than one" नहीं
(E) गलत: B सही है, इसलिए "None" नहीं

Ans: (B) SOP

Y = AB + BC + AC में AB, BC, AC तीन products (AND) हैं, इन्हें + से जोड़ा गया है. इसे Sum of Products (SOP (Sum of Products)) कहते हैं. जवाब (B).

Q2. [BPSC TRE 1.0 (2023)]

What are the canonical forms of Boolean expressions?

(A) OR and XOR
(B) NOR and XNOR
(C) SOM and POM
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: OR और XOR (Exclusive OR) gates के नाम हैं, canonical forms नहीं
(B) गलत: NOR (NOT OR) और XNOR (Exclusive NOR) भी gates हैं
(C) ✔ सही: Canonical forms दो हैं: SOM (Sum of Minterms) और POM (Product of Maxterms)
(D) गलत: सिर्फ C सही है
(E) गलत: C सही है

Ans: (C) SOM and POM

Canonical form यानी हर term में सारे variables मौजूद हों. दो तरह: SOM (Sum of Minterms, हर term एक AND of all variables) और POM (Product of Maxterms, हर term एक OR of all variables). जवाब (C).

Q3. [BPSC TRE 1.0 (2023)]

Boolean algebra can be used

(A) for designing the digital computers
(B) in building logic symbols
(C) in circuit theory
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Boolean algebra से digital computer के circuits डिज़ाइन होते हैं
(B) ✔ सही: Logic symbols (gates) के circuits बनाने में इसका उपयोग होता है
(C) ✔ सही: Circuit theory (switching circuits) में भी इसका उपयोग है
(D) ✔ सही: A, B, C तीनों सही हैं, इसलिए "More than one of the above"
(E) गलत: तीनों सही हैं, इसलिए "None" नहीं

Ans: (D) More than one of the above

Boolean algebra (George Boole की बनाई 0/1 की algebra) से computer का हर logic circuit बनता और सरल होता है. तीनों बातें सही हैं, इसलिए "More than one of the above". जवाब (D).

Q4. [BPSC TRE 2.0 (2023)]

The Boolean expression x + x′y equals

(A) x + y
(B) x + xy
(C) y + yx
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: x + x'y = x + y. (x+x')(x+y) = 1.(x+y)
(B) गलत: x + xy = x होता है (absorption), यह x+y नहीं
(C) गलत: y + yx = y होता है
(D) गलत: सिर्फ A सही है
(E) गलत: A सही है

Ans: (A) x + y

Rule (absorption का दूसरा रूप): x + x'y = x + y. सीधा test: x=0,y=1: मूल = 0 + 1.1 = 1, x+y = 1 मेल खाता है. x=0,y=0: मूल 0, x+y = 0 मेल. जवाब (A).

Step-by-step Solution:

1. Distributive law से x + x'y = (x + x')(x + y).
2. x + x' = 1, तो = 1.(x + y) = x + y.
3. Answer (A). Trap: x + xy = x (यह अलग rule है).

Q5. [BPSC TRE 2.0 (2023)]

The Boolean expression AB + AB′ + A′C + AC is independent of Boolean variable

(A) A
(B) B
(C) C
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: A पर function निर्भर है (A खत्म करने पर बदलता है)
(B) ✔ सही: Simplify करने पर AB के term खत्म हो जाते हैं और F = A + C बचता है. B कहीं नहीं
(C) गलत: C पर निर्भर है
(D) गलत: सिर्फ B ही independent है
(E) गलत: B सही है

Ans: (B) B

F = AB + AB' + A'C + AC. पहले दो: A(B + B') = A. आख़िरी दो: C(A' + A) = C. तो F = A + C. इसमें B है ही नहीं, इसलिए function B से independent है. जवाब (B).

Step-by-step Solution:

1. F = AB + AB' + A'C + AC.
2. पहले दो terms: AB + AB' = A(B + B') = A.1 = A.
3. आख़िरी दो terms: A'C + AC = C(A' + A) = C.1 = C.
4. F = A + C. इसमें B नहीं है, इसलिए F, B से independent है. Answer (B).

Q6. [BPSC TRE 2.0 (2023)]

The Boolean expression A + BC equals

(A) (A′ + B)(A′ + C)
(B) (A + B)(A′ + C)
(C) (A + B)(A + C)
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: (A'+B)(A'+C) = A' + BC होगा, A + BC नहीं
(B) गलत: (A+B)(A'+C) = AC + A'B जैसा कुछ बनेगा
(C) ✔ सही: Distributive law: A + BC = (A+B)(A+C). यह Boolean algebra की ख़ास बात है, साधारण algebra में नहीं चलती
(D) गलत: सिर्फ C सही है
(E) गलत: C सही है

Ans: (C) (A + B)(A + C)

Boolean algebra में OR का AND पर distribute भी होता है: A + BC = (A+B)(A+C). Proof: (A+B)(A+C) = AA + AC + AB + BC = A + AC + AB + BC = A(1 + C + B) + BC =

A + BC. जवाब (C).

Step-by-step Solution:

1. Right side खोलो: (A+B)(A+C) = A.A + A.C + B.A + B.C.
2. A.A = A. तो = A + AC + AB + BC.
3. A + AC + AB = A(1 + C + B) = A.1 = A.
4. बचा A + BC, जो left side है. इसलिए A + BC = (A+B)(A+C). Answer (C).

Q7. [BPSC TRE 2.0 (2023)]

The Boolean expression x + xy will be equal to

(A) x
(B) y
(C) x + y
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: x + xy = x(1 + y) = x. यह absorption law है
(B) गलत: y नहीं, x रहता है
(C) गलत: x + y तब होता जब x'y होता (x + x'y = x + y)
(D) गलत: सिर्फ A सही है
(E) गलत: A सही है

Ans: (A) x

x + xy = x(1 + y). 1 + y हमेशा 1 होता है. इसलिए x.1 = x. इसे Absorption law कहते हैं (xy को x ने सोख लिया). जवाब (A). Trap: x + x'y और x + xy को मत मिलाओ.

Q8. [BPSC TRE 2.0 (2023)]

The Boolean expression x(x + y) is equal to

(A) x
(B) y
(C) xy
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: x(x + y) = xx + xy = x + xy = x. यह absorption law है
(B) गलत: y नहीं बचता
(C) गलत: xy तब मिलता जब bracket में x न होता
(D) गलत: सिर्फ A सही है
(E) गलत: A सही है

Ans: (A) x

x(x + y) = x.x + x.y = x + xy = x(1 + y) = x. यह absorption law का दूसरा रूप (AND-OR वाला) है. जवाब (A).

Q9. [BPSC TRE 2.0 (2023)]

If x is a Boolean variable, then (((x′)′)′)′ will be

(A) x′
(B) x
(C) x + x′
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: चार बार NOT करने पर x' नहीं, x बनता है
(B) ✔ सही: हर दो NOT आपस में कट जाते हैं (double negation): चार NOT = दो जोड़ी = x
(C) गलत: x + x' = 1 है, यह x नहीं
(D) गलत: सिर्फ B सही है
(E) गलत: B सही है

Ans: (B) x

(x')' = x (double negation). सबसे अंदर से चलो: x' ; (x')' = x ; ((x')')' = x' ; (((x')')')' = x. चार NOT बराबर 0 NOT. Trick: NOT की संख्या सम (even) तो x, विषम (odd) तो x'. जवाब (B).

Q10. [BPSC TRE 2.0 (2023)]

In two-valued Boolean algebra, the maximum number of Boolean functions for two variables will be

(A) 8
(B) 12
(C) 16
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: 8 बहुत कम है, यह 3 variables की truth table की rows की संख्या है
(B) गलत: 12 किसी formula से नहीं आता
(C) ✔ सही: 2 variables की 4 rows, हर row का output 0 या 1, कुल 2^4 = 16 functions
(D) गलत: सिर्फ C सही है
(E) गलत: C सही है

Ans: (C) 16

n variables के लिए truth table की rows = 2^n. हर row का output 0 या 1 हो सकता है, तो कुल functions = 2^(2^n). n=2: rows = 4, functions = 2^4 = 16. जवाब (C). Trap: 2^2 = 4 पर रुक जाना.

Step-by-step Solution:

1. Variables n = 2.
2. Truth table की rows = 2^n = 2^2 = 4.
3. हर row का output दो तरह का (0 या 1), तो कुल functions = 2^(rows) = 2^4 = 16.
4. Formula: 2^(2^n). Answer (C). Trap: 2^n = 4 को answer मान लेना (वो rows हैं).

Q11. [BPSC TRE 2.0 (2023)]

Simplified Boolean function of Boolean function F = A′C + A′B + AB′C + BC is

(A) F = C + A′B
(B) F = A′B + B′C
(C) F = BC + A′B
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: A'C + AB'C = C(A' + B') और बाकी जोड़ने पर F = C + A'B बचता है
(B) गलत: A'B + B'C में BC वाला term और C का अकेला term खो जाता है
(C) गलत: BC + A'B अधूरा है, C अकेला नहीं है
(D) गलत: सिर्फ A सही है
(E) गलत: A सही है

Ans: (A) F = C + A′B

F = A'C + A'B + AB'C + BC. A'C + AB'C = C(A' + AB') = C(A' + B') (rule: x + x'y = x + y). अब F = A'C + B'C + A'B + BC. B'C + BC = C. तो F = C + A'C + A'B = C + A'B. जवाब (A).

Step-by-step Solution:

1. F = A'C + A'B + AB'C + BC.
2. A'C + AB'C = C(A' + AB'). x + x'y = x + y वाला rule (यहाँ x = A' और x' = A): A' + AB' = A' + B'. तो = C(A' + B') = A'C + B'C.
3. अब F = A'C + B'C + A'B + BC.
4. B'C + BC = C(B' + B) = C.
5. F = C + A'C + A'B. C + A'C = C. तो F = C + A'B.
6. Answer (A). Check: A=1,B=1,C=0: मूल = 0+0+0+0 = 0, C + A'B = 0 + 0 = 0 ठीक.

Q12. [BPSC TRE 2.0 (2023)]

Which of the following logic gates is called universal gate?

(A) AND
(B) OR
(C) NAND
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: AND से NOT नहीं बन सकता, universal नहीं
(B) गलत: OR से NOT नहीं बन सकता, universal नहीं
(C) ✔ सही: NAND (NOT AND) universal gate है, इससे NOT, AND, OR सब बनते हैं
(D) गलत: सिर्फ C सही है
(E) गलत: C सही है

Ans: (C) NAND

दो ही universal gates हैं: NAND (NOT AND) और NOR (NOT OR). यहाँ options में सिर्फ NAND है. NAND से NOT: NAND(A,A) = A'. जवाब (C).

Q13. [BPSC TRE 2.0 (2023)]

The minimum number of NAND gates required to implement Boolean function f = x′y + xy′ is

(A) 3
(B) 4
(C) 6
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: 3 gates से XOR (Exclusive OR) नहीं बनता
(B) ✔ सही: XOR = x'y + xy' को NAND (NOT AND) से बनाने में 4 NAND लगते हैं (standard design)
(C) गलत: 6 gates ज़्यादा हैं, यह पहली सोची गई विधि है, minimum नहीं
(D) गलत: सिर्फ B सही है
(E) गलत: B सही है

Ans: (B) 4

XOR (Exclusive OR) को 4 NAND (NOT AND) gates से ऐसे बनाते हैं: N1 = NAND(x,y); N2 = NAND(x,N1); N3 = NAND(y,N1); Output = NAND(N2,N3). चार gates, यही minimum है. जवाब (B).

Step-by-step Solution:

1. N1 = NAND(x, y) = (xy)'.
2. N2 = NAND(x, N1) = (x.(x' + y'))' = (xy')' = x' + y.
3. N3 = NAND(y, N1) = y' + x.
4. Output = NAND(N2, N3) = (N2.N3)' = ((x' + y)(y' + x))'.
5. (x'+y)(y'+x) = x'y' + xy (XNOR (Exclusive NOR)). इसका NOT = XOR (Exclusive OR) = x'y + xy'.
6. कुल 4 NAND (NOT AND) gates. Answer (B).

Q14. [BPSC TRE 3.0 (2024)]

Digital circuit can be made by the repeated use of the _____.

(A) NAND gate
(B) NOR gate
(C) AND gate
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: NAND (NOT AND) universal है, इससे हर digital circuit बनता है
(B) ✔ सही: NOR (NOT OR) भी universal है, इससे भी हर circuit बनता है
(C) गलत: सिर्फ AND से NOT नहीं बन सकता, इसलिए पूरा circuit नहीं बनता
(D) ✔ सही: A और B दोनों सही हैं, इसलिए "More than one of the above"
(E) गलत: A और B सही हैं

Ans: (D) More than one of the above

Universal gate: NAND (NOT AND) और NOR (NOT OR). दोनों को बार-बार लगाकर कोई भी digital circuit बन सकता है. यहाँ दो सही options हैं, इसलिए जवाब (D) More than one of the above.

Q15. [BPSC TRE 3.0 (2024)]

Which of the following is correct for Digital Circuits?

(A) They use analog signals for communication
(B) They process information using continuous voltage levels
(C) They are not suitable for high speed operations
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: Digital circuit digital (discrete 0/1) signals पर चलते हैं, analog पर नहीं
(B) गलत: Digital में voltage के सिर्फ दो levels (low/high) होते हैं, continuous नहीं
(C) गलत: Digital circuits तेज़ होते हैं, high speed के लिए सही हैं
(D) गलत: A, B, C तीनों गलत हैं, इसलिए "More than one" नहीं
(E) ✔ सही: तीनों statements गलत हैं, इसलिए "None of the above"

Ans: (E) None of the above

Digital circuit 0 और 1 (दो voltage levels) पर काम करते हैं, तेज़ होते हैं और noise से कम बिगड़ते हैं. तीनों statements उल्टी हैं, इसलिए जवाब (E) None of the above. Trap: "Which is correct" में सब गलत भी हो सकते हैं.

Q16. [BPSC TRE 3.0 (2024)]

Which of the following is an example of a digital Electronic?

(A) Computers
(B) Mobile phones
(C) Digital cameras
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Computer digital electronic device है
(B) ✔ सही: Mobile phone भी digital electronics से चलता है
(C) ✔ सही: Digital camera भी digital है
(D) ✔ सही: तीनों सही हैं, इसलिए "More than one of the above"
(E) गलत: तीनों सही हैं

Ans: (D) More than one of the above

Computer, mobile, digital camera, calculator, digital watch सब digital electronics के उदाहरण हैं. तीनों सही हैं, इसलिए जवाब (D).

Q17. [BPSC TRE 3.0 (2024)]

An OR gate has 4 inputs. The number of words in truth table will be _____.

(A) 4
(B) 8
(C) 16
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: 4 तो inputs की संख्या है, rows की नहीं
(B) गलत: 8 तब आता जब 3 inputs होते
(C) ✔ सही: 4 inputs के लिए rows = 2^4 = 16
(D) गलत: सिर्फ C सही है
(E) गलत: C सही है

Ans: (C) 16

Truth table में हर input combination एक row (word) है. n inputs के लिए rows = 2^n. OR gate हो या AND gate, rows की संख्या वही रहती है. 2^4 = 16. जवाब (C).

Step-by-step Solution:

1. Inputs n = 4.
2. Rows = 2^n = 2 x 2 x 2 x 2 = 16.
3. Gate का प्रकार (OR) rows की संख्या नहीं बदलता, सिर्फ output column बदलता है.
4. Answer (C). Trap: 4 inputs देखकर 4 चुन लेना.

Q18. [BPSC TRE 3.0 (2024)]

When does a negative level triggered flip-flop in Digital Electronics changes its state?

(A) When the clock is Negative
(B) When the clock is Positive
(C) When the inputs are all zero
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Negative level triggered flip-flop तब बदलता है जब clock negative (low, 0) level पर हो
(B) गलत: Positive level पर तो positive level triggered flip-flop बदलता है
(C) गलत: Inputs 0 होने का इससे लेना-देना नहीं, trigger clock से तय होता है
(D) गलत: सिर्फ A सही है
(E) गलत: A सही है

Ans: (A) When the clock is Negative

Clock एक 0-1-0-1 की पल्स है. Flip-flop को clock से "trigger" (इशारा) मिलता है. Level triggered: clock जब तक एक level पर है तब तक बदल सकता है. Positive = clock 1 पर, Negative = clock 0 पर. जवाब (A). Trap: edge triggered (clock के बदलने के क्षण पर) अलग होता है.

Q19. [BPSC TRE 3.0 (2024)]

What is the result of the operation 1010 AND 1100 in binary?

(A) 1000
(B) 1110
(C) 100
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: 1010 AND 1100 बिट-दर-बिट: 1,0,0,0 -> 1000
(B) गलत: 1110 वह OR का नतीजा है, AND का नहीं
(C) गलत: 100 सिर्फ तीन bit का है, ठीक रूप 1000 है
(D) गलत: सिर्फ A सही है
(E) गलत: A सही है

Ans: (A) 1000

AND में output तभी 1 होता है जब दोनों bit 1 हों. Bit-by-bit: 1&1 = 1, 0&1 = 0, 1&0 = 0, 0&0 = 0. नतीजा 1000. जवाब (A). C++ में यह & operator है: 10 & 12 = 8.

Step-by-step Solution:

1. 1010 AND 1100, बाएँ से दाएँ bit-by-bit.
2. पहला bit: 1 AND 1 = 1.
3. दूसरा bit: 0 AND 1 = 0.
4. तीसरा bit: 1 AND 0 = 0.
5. चौथा bit: 0 AND 0 = 0.
6. Result = 1000 (decimal 8). Answer (A). Check: 10 & 12 = 8. Trap: OR करने पर 1110 आता है.

Q20. [BPSC TRE 3.0 (2024)]

In a half-adder circuit, what are the outputs?

(A) Sum and Carry
(B) Difference and Borrow
(C) Sum and Difference
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Half adder के दो outputs हैं: Sum (A XOR (Exclusive OR) B) और Carry (A AND B)
(B) गलत: Difference और Borrow half subtractor के outputs हैं
(C) गलत: Sum और Difference साथ में कोई adder/subtractor नहीं देता
(D) गलत: सिर्फ A सही है
(E) गलत: A सही है

Ans: (A) Sum and Carry

Half adder दो 1-bit संख्याएँ जोड़ता है. 0+0=00, 0+1=01, 1+0=01, 1+1=10 (यानी sum 0 और carry 1). Sum = A XOR (Exclusive OR) B, Carry = A AND B. जवाब (A). Half subtractor के outputs Difference और Borrow होते हैं.

Q21. [BPSC TRE 3.0 (2024)]

What is De Morgan’s theorem used for in digital logic?

(A) Simplifying Boolean expressions
(B) Implementing NAND gates
(C) Performing binary addition
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: De Morgan के नियमों से expressions simplify (सरल) किए जाते हैं
(B) ✔ सही: AND/OR को NAND (NOT AND)/NOR (NOT OR) में बदलने के लिए De Morgan चाहिए, इसलिए NAND gates से implement करने में काम आता है
(C) गलत: Binary addition adders से होती है, De Morgan से नहीं
(D) ✔ सही: A और B दोनों सही, इसलिए "More than one of the above"

(E) गलत: A और B सही हैं

Ans: (D) More than one of the above

De Morgan: (A.B)' = A' + B' और (A+B)' = A'.B'. इससे (1) expressions सरल होते हैं, (2) AND-OR circuit को NAND-NAND (NOT AND) या NOR-NOR (NOT OR) में बदलना आसान होता है. बाइनरी जोड़ का इससे संबंध नहीं. जवाब (D).

Q22. [UGC NET Dec 2004]

The Boolean identity A + A = A (A OR A = A) is called :

(A) Identity law
(B) De Morgan’ s law
(C) Idempotent law
(D) Complement law

हर **option** समझो:

(A) गलत: Identity law में A+0=A या A.1=A आता है, A+A नहीं
(B) गलत: De Morgan law (डी मॉर्गन नियम) NOT को AND/OR के अंदर बाँटता है: (A+B)'=A'.B'
(C) ✔ सही: Idempotent law (इडेम्पोटेंट नियम): अपने साथ OR/AND करने पर वही रहता है, A+A=A और A.A=A
(D) गलत: Complement law में A+A'=1 और A.A'=0 आता है; यहाँ दोनों A एक जैसे हैं

Ans: (C) Idempotent law

A+A का मतलब है "A या A". अगर A=1 तो 1 या 1 = 1, और A=0 तो 0 या 0 = 0. हर हाल में जवाब A ही है. इसी को Idempotent law कहते हैं. जवाब (C).

Q23. [UGC NET Dec 2004]

An example of a connective which is not associative is :

(A) AND
(B) OR
(C) EX-OR
(D) NAND

हर **option** समझो:

(A) गलत: AND associative है, (A.B).C = A.(B.C)
(B) गलत: OR भी associative है, (A+B)+C = A+(B+C)
(C) गलत: XOR (Exclusive OR) भी associative है, बहुत लोग इसे गलत मान लेते हैं
(D) ✔ सही: NAND (NOT AND) में (A NAND (NOT AND) B) NAND C, A NAND (B NAND C) से अलग आ सकता है

Ans: (D) NAND

Associative का मतलब: तीन चीज़ों को किसी भी जोड़ी में group करो, जवाब वही रहे. AND, OR, XOR (Exclusive OR) में यह सही है. NAND (NOT AND) और NOR (NOT OR) में नहीं. Test: A=0,B=0,C=1 रखो: (0 NAND 0)=1, फिर 1 NAND 1 = 0. दूसरी तरफ़ (0 NAND 1)=1, फिर 0 NAND 1 = 1. दोनों अलग हैं. जवाब (D).

Step-by-step Solution:

1. A=0, B=0, C=1 लो. Left: (A NAND (NOT AND) B) = NOT(0.0) = 1.
2. फिर (1 NAND C) = NOT(1.1) = 0.
3. Right: (B NAND C) = NOT(0.1) = 1. फिर A NAND 1 = NOT(0.1) = 1.
4. Left = 0 और Right = 1, दोनों अलग. इसलिए NAND associative नहीं है. Answer (D).

Q24. [UGC NET Dec 2004]

The characteristic equation of a T flip-flop is : [Note: the symbols used have the usual meaning]

(A) Q(n+1) = T XOR Q(n)
(B) Q(n+1) = T + Q(n)
(C) Q(n+1) = T . Q(n)
(D) Q(n+1) = T Q(n) [bar missing in source]

हर **option** समझो:

(A) ✔ सही: T flip-flop (Toggle) में T=1 पर output पलटता है, T=0 पर वही रहता है. यही XOR (Exclusive OR) है: Q+ = T XOR Q
(B) गलत: T+Q (OR) में T=1 पर output हमेशा 1 रहेगा, पलटेगा नहीं
(C) गलत: T.Q (AND) में T=1 पर Q वही रहेगा, पलटेगा नहीं; T=0 पर 0 हो जाएगा
(D) गलत: यह T.Q' जैसी चीज़ है, toggle की पूरी शर्त नहीं देती

Ans: (A) Q(n+1) = T XOR Q(n)

T का मतलब Toggle (पलटना). T=0 पर output नहीं बदलता, T=1 पर उल्टा हो जाता है. Truth table: T=0,Q=0 -> 0; T=0,Q=1 -> 1; T=1,Q=0 -> 1; T=1,Q=1 -> 0. यही XOR (T XOR (Exclusive OR) Q) की table है. जवाब (A).

Q25. [UGC NET Jun 2005]

Which of the following logic has the maximum fan out ?

(A) RTL
(B) ECL
(C) NMOS
(D) CMOS

हर **option** समझो:

(A) गलत: RTL (Resistor Transistor Logic) पुरानी family है, fan out बहुत कम
(B) गलत: ECL (Emitter Coupled Logic) सबसे तेज़ है, पर fan out CMOS (Complementary Metal Oxide Semiconductor) से कम
(C) गलत: NMOS (N-type Metal Oxide Semiconductor) का fan out अच्छा है पर CMOS से कम
(D) ✔ सही: CMOS (Complementary MOS (Metal Oxide Semiconductor)) का input current लगभग शून्य होता है, इसलिए fan out सबसे ज़्यादा

Ans: (D) CMOS

Fan out यानी एक gate का output कितने दूसरे gates के input चला सकता है. CMOS (Complementary Metal Oxide Semiconductor) के input पर current लगभग नहीं खिंचता,

इसलिए एक output से बहुत सारे gate जुड़ सकते हैं. CMOS का fan out सबसे ज़्यादा है. जवाब (D).

Q26. [UGC NET Dec 2005]

A half-adder is also known as :

(A) AND Circuit
(B) NAND Circuit
(C) NOR Circuit
(D) EX-OR Circuit

हर **option** समझो:

(A) गलत: AND सिर्फ carry देता है, sum नहीं
(B) गलत: NAND (NOT AND) half adder नहीं है
(C) गलत: NOR (NOT OR) half adder नहीं है
(D) ✔ सही: Half adder का sum = A XOR (Exclusive OR) B है, इसलिए इसे EX-OR circuit कहा गया (carry = A.B अलग से)

Ans: (D) EX-OR Circuit

Half adder दो 1-bit संख्याएँ जोड़ता है. Sum = A XOR (Exclusive OR) B और Carry = A AND B. इस सवाल में sum वाले मुख्य हिस्से के कारण key "EX-OR circuit" है. Trap: असल में half adder = XOR + AND दोनों gates. जवाब (D).

Q27. [UGC NET Dec 2005]

Consider the following sequence of instructions :

a = a XOR b, b = a XOR b, a = b XOR a

This sequence :

(A) retains the value of the a and b
(B) complements the value of a and b
(C) swap a and b
(D) negates values of a and b

हर **option** समझो:

(A) गलत: Values वैसी की वैसी नहीं रहतीं, a और b की जगह बदल जाती है
(B) गलत: Complement (उल्टा) तब होता है जब हर bit पलटे; यहाँ ऐसा नहीं होता
(C) ✔ सही: तीन XOR (Exclusive OR) से a और b की values आपस में swap (अदला-बदली) हो जाती हैं, बिना तीसरे variable के
(D) गलत: Negate का मतलब -a, -b होता है, XOR से यह नहीं होता

Ans: (C) swap a and b

यह XOR (Exclusive OR) swap trick है. मान लो a=A0, b=B0. Step 1: a = A0 XOR B0. Step 2: b = (A0 XOR B0) XOR B0 = A0. Step 3: a = b XOR a = A0 XOR (A0 XOR B0) = B0. तो अब a=B0 और b=A0 है, यानी swap. Bank key (A) गलत लगती है, क्योंकि step 2 में a नई value (A0 XOR B0) है. जवाब (C).

Step-by-step Solution:

1. शुरू में a = 5 (0101), b = 3 (0011) लो.

2. Step 1: a = a XOR (Exclusive OR) b = 0101 XOR 0011 = 0110 (6).
3. Step 2: b = a XOR b = 0110 XOR 0011 = 0101 (5). अब b = 5 (पुरानी a).
4. Step 3: a = b XOR a = 0101 XOR 0110 = 0011 (3). अब a = 3 (पुरानी b).
5. a=3, b=5 मिला, यानी दोनों swap हुए. Answer (C). Trap: "पहली line के a की value" मत पकड़ो, हर line में a की ताज़ा value लो.

Q28. [UGC NET Dec 2005]

When an inverter is placed between both inputs of an S-R flip flop, the resulting flip flop is :

(A) JK flip-flop
(B) D-flip-flop
(C) T flip-flop
(D) None of these

हर **option** समझो:

(A) गलत?: Inverter सिर्फ एक input को उल्टा करता है तो JK नहीं बनता, JK को feedback लगता है
(B) गलत?: D flip-flop तब बनता है जब S और R के बीच inverter ठीक तरह जुड़े (S=D, R=D')
(C) गलत?: T flip-flop के लिए feedback चाहिए, सिर्फ inverter से नहीं बनता
(D) ✔ सही (key): सवाल अधूरा है, inverter कहाँ लगा यह नहीं बताया, इसलिए key "None of these"

Ans: (D) None of these

Standard किताबों में S और R के बीच एक inverter लगाने से D flip-flop बनता है. पर इस सवाल में "inverter कहाँ जुड़ा" साफ़ नहीं है, इसलिए source ने सवाल को ambiguous (अस्पष्ट) माना और key (D) None of these दी. Exam में अगर "S और R के बीच inverter" साफ़ लिखा हो तो D flip-flop चुनो. जवाब (D) key के अनुसार.

Q29. [UGC NET Jun 2006]

Upto how many variables, can the Karnaugh map be used ?

(A) 3
(B) 4
(C) 5
(D) 6

हर **option** समझो:

(A) गलत: 3 variable तो बहुत आसान map है, इससे ज़्यादा भी चलता है
(B) गलत: 4 variable (16 cells) सबसे आम है, पर limit नहीं
(C) गलत: 5 variable (32 cells) भी बनता है, पर limit नहीं
(D) ✔ सही: K-map (Karnaugh map) 6 variable तक चलता है; उससे ज़्यादा में cells बहुत हो जाते हैं

Ans: (D) 6

K-map (Karnaugh map) में cells की संख्या 2 की power होती है: n variable के लिए 2^n. 4 variable = 16 cells, 5 = 32, 6 = 64. इसके बाद hand से बनाना मुश्किल हो जाता है, तो Quine-McCluskey जैसी method लेते हैं. Exam key: 6 variable. जवाब (D).

Q30. [UGC NET Dec 2006]

A sum of products expression can be implemented with _____ logic gates.

(A) AND − OR
(B) NAND − OR
(C) AND − NOT
(D) OR − AND

हर **option** समझो:

(A) ✔ सही: SOP (Sum of Products) = कई AND के output को एक OR में जोड़ना. यानी AND-OR logic
(B) गलत: NAND-OR mix SOP (Sum of Products) का standard रूप नहीं है (NAND-NAND (NOT AND) होता है)
(C) गलत: AND-NOT से OR नहीं बनता, पूरा function नहीं बनेगा
(D) गलत: OR-AND logic तो POS (Product of Sums) के लिए है

Ans: (A) AND − OR

SOP (Sum of Products) का अर्थ: पहले products (AND), फिर उनका sum (OR). जैसे F = AB + CD: पहले AB और CD के AND gates, फिर दोनों को OR gate. इसलिए AND-OR. उल्टा POS (Product of Sums) में OR-AND लगता है. जवाब (A).

Q31. [UGC NET Dec 2006]

Which of the following logic is the fastest ?

(A) RTL
(B) ECL
(C) HTL
(D) HCL

हर **option** समझो:

(A) गलत: RTL (Resistor Transistor Logic) धीमी family है
(B) ✔ सही: ECL (Emitter Coupled Logic) में transistor saturation में नहीं जाते, इसलिए सबसे तेज़
(C) गलत: HTL (High Threshold Logic) शोर वाली जगहों के लिए है, धीमी है
(D) गलत: HCL इस नाम की कोई मानक तेज़ family नहीं है

Ans: (B) ECL

Transistor को पूरा ON (saturation) करने पर उसे बंद होने में समय लगता है. ECL (Emitter Coupled Logic) में transistor saturation में जाते ही नहीं, इसलिए switching सबसे तेज़ है. पर इसकी power ज़्यादा लगती है. जवाब (B).

Q32. [UGC NET Jun 2007]

What is the maximum counting speed of a 4-bit binary counter which is composed of Flip-Flop with a propagation delay of 25ns ?

(A) 1MHz
(B) 10MHz
(C) 100MHz
(D) 4MHz

हर **option** समझो:

(A) गलत: 1 MHz (Mega Hertz) बहुत कम निकलेगा, यह तब आता जब delay 250 ns होता
(B) ✔ सही: 4 flip-flop x 25 ns = 100 ns कुल delay; f = 1/100ns = 10 MHz
(C) गलत: 100 MHz तब आता जब पूरी counter की delay 10 ns होती
(D) गलत: 4 MHz का कोई सीधा हिसाब नहीं बनता (1/25ns = 40 MHz, 4 से गुणा करने वाली गलती)

Ans: (B) 10MHz

Ripple (asynchronous) counter में हर flip-flop अपनी बारी पर बदलता है, तो सबकी delay जुड़ती है. Total delay = n x tpd (propagation delay). Maximum frequency = 1 / total delay. Trap: 1/25ns = 40 MHz (Mega Hertz) लेकर रुक जाना गलत है.

Step-by-step Solution:

1. Formula: Maximum counting frequency (f_max) = 1 / (n x tpd (propagation delay)), जहाँ n = flip-flops की संख्या, tpd = propagation delay (Propagation Delay).
2. Values: n = 4, tpd = 25 ns.
3. Total delay = 4 x 25 ns = 100 ns.
4. f_max = 1 / (100 x 10^-9 s) = 10^7 Hz = 10 MHz (Mega Hertz).
5. Answer (B). Trap: 1/25 ns = 40 MHz लेना (सिर्फ एक flip-flop की delay).

Q33. [UGC NET Dec 2007]

Simplified form of Boolean expression xy+(~x)z+yz is :

(A) xy+(~x)z.
(B) (~x)y+(~x)z.
(C) (~x)y+xz.
(D) xy+xz.

हर **option** समझो:

(A) ✔ सही: xy + x'z + yz में तीसरा term yz बेकार (redundant) है, इसे consensus theorem हटा देता है. बचा xy + x'z
(B) गलत: x'y + x'z नहीं बनता, xy term गायब नहीं होता
(C) गलत: x'y + xz बनाने में x और x' की जगह उलट दी गई है
(D) गलत: xy + xz में x'z का term खो गया है

Ans: (A) xy+(~x)z.

Consensus theorem: xy + x'z + yz = xy + x'z. तीसरा term yz पहले दो terms का

'consensus' है, इसलिए हट जाता है. जवाब (A).

Step-by-step Solution:

1. दिया: F = xy + x'z + yz.
2. yz को (x + x') से गुणा करो (क्योंकि x + x' = 1): yz = yz.x + yz.x' = xyz + x'yz.
3. अब F = xy + x'z + xyz + x'yz.
4. xy + xyz = xy(1 + z) = xy. और x'z + x'yz = x'z(1 + y) = x'z.
5. तो F = xy + x'z. Answer (A).

Q34. [UGC NET Dec 2007]

In order to build a MOD-18 counter, the minimum number of flip flops needed is equal to :

(A) 18
(B) 9
(C) 5
(D) 4

हर **option** समझो:

(A) गलत: 18 flip-flop लगाना जरूरी नहीं, n flip-flop से 2^n states मिलती हैं
(B) गलत: 9 flip-flop से 512 states मिलतीं, बहुत ज़्यादा
(C) ✔ सही: 4 flip-flop से सिर्फ 16 states (कम), 5 से 32 states (काफ़ी). तो न्यूनतम 5
(D) गलत: 4 flip-flop से 16 states मिलतीं, MOD-18 के लिए 18 चाहिए, कम पड़ेगा

Ans: (C) 5

MOD-N counter का मतलब N states तक गिनना. n flip-flop 2^n states दे सकते हैं. हमें 2^n >= N चाहिए. 2^4 = 16 < 18, 2^5 = 32 >= 18. इसलिए 5 flip-flop. जवाब (C).

Step-by-step Solution:

1. Condition: 2^n >= N, जहाँ N = MOD (modulus) = 18.
2. n = 4: 2^4 = 16, 16 < 18 इसलिए काफ़ी नहीं.
3. n = 5: 2^5 = 32, 32 >= 18 इसलिए काफ़ी.
4. Minimum flip-flops = 5. Answer (C). Trap: N को ही flip-flops की संख्या मान लेना.

Q35. [UGC NET Dec 2007]

Amongst the logic families DTL, TTL, ECL and CMOS, the family with the least power dissipation is :

(A) CMOS
(B) DTL
(C) TTL
(D) ECL

हर **option** समझो:

(A) ✔ सही: CMOS (Complementary Metal Oxide Semiconductor) में कोई एक transistor हमेशा बंद रहता है, तो static power लगभग शून्य (nW स्तर)
(B) गलत: DTL (Diode Transistor Logic) में काफ़ी power लगती है

(C) गलत: TTL (Transistor-Transistor Logic) में mW स्तर की power लगती है
(D) गलत: ECL (Emitter Coupled Logic) में transistor हमेशा चालू रहते हैं, power सबसे ज़्यादा

Ans: (A) CMOS

CMOS (Complementary Metal Oxide Semiconductor) में NMOS (N-type Metal Oxide Semiconductor) और PMOS जोड़ी में रहते हैं, एक ON तो दूसरा OFF. इसलिए steady state में current लगभग नहीं बहता. Power सिर्फ switching के वक्त लगती है. इसलिए सबसे कम power CMOS में. जवाब (A).

Q36. [UGC NET Jun 2008]

Among the logic families RTL, TTL, ECL and CMOS, the fastest family is :

(A) ECL
(B) CMOS
(C) TTL
(D) RTL

हर **option** समझो:

(A) ✔ सही: ECL (Emitter Coupled Logic) सबसे तेज़, propagation delay कुछ ns से भी कम
(B) गलत: CMOS (Complementary MOS (Metal Oxide Semiconductor)) कम power वाली है, गति में ECL (Emitter Coupled Logic) से पीछे
(C) गलत: TTL (Transistor-Transistor Logic) मध्यम गति (लगभग 10 ns)
(D) गलत: RTL (Resistor Transistor Logic) सबसे धीमी पुरानी family

Ans: (A) ECL

याद रखने की लाइन: Fastest = ECL (Emitter Coupled Logic); Least power = CMOS (Complementary Metal Oxide Semiconductor); Highest fan out = CMOS. ECL इसलिए तेज़ कि transistor saturation में नहीं जाते. जवाब (A).

Q37. [UGC NET Jun 2008]

The characteristic equation of a T flip flop is given by :

(A) Q(N+1) = T . Q(N)
(B) Q(N+1) = T + Q(N)
(C) Q(N+1) = T XOR Q(N)
(D) Q(N+1) = T' . Q(N)

हर **option** समझो:

(A) गलत: T.Q (AND) toggle नहीं करता. T=1 पर Q वैसा ही रहेगा
(B) गलत: T+Q (OR) T=1 पर हमेशा 1 देगा, पलटेगा नहीं
(C) ✔ सही: T XOR (Exclusive OR) Q: T=0 पर Q वही, T=1 पर Q उल्टा. यही toggle है
(D) गलत: T'.Q कभी भी Q को 0 से 1 नहीं कर सकता, toggle नहीं है

Ans: (C) Q(N+1) = T XOR Q(N)

T flip-flop का characteristic equation Q(N+1) = T XOR (Exclusive OR) Q(N) = T'Q + TQ'. Bank की key (A) हमें source की छपाई की गलती (bar गायब) लगती है; ऊपर के options में

सिर्फ XOR वाला option logic से सही है. (cs0010 में भी यही equation है.) जवाब (C).

Q38. [UGC NET Dec 2008]

Extremely low power dissipation and low cost per gate can be achieved in :

(A) MOS ICs
(B) C MOS ICs
(C) TTL ICs
(D) ECL ICs

हर **option** समझो:

(A) गलत: MOS (Metal Oxide Semiconductor) ICs (Integrated Circuits) में power कम है पर CMOS (Complementary Metal Oxide Semiconductor) जितनी नहीं
(B) ✔ सही: CMOS ICs (Integrated Circuits) में बहुत कम power और कम cost per gate (छोटा size) मिलता है
(C) गलत: TTL (Transistor-Transistor Logic) में power ज़्यादा लगती है
(D) गलत: ECL (Emitter Coupled Logic) सबसे ज़्यादा power लेती है और महँगी भी है

Ans: (B) C MOS ICs

दो शर्तें हैं: बहुत कम power और कम cost per gate. CMOS (Complementary Metal Oxide Semiconductor) दोनों देती है. जवाब (B).

Q39. [UGC NET Dec 2008]

An example of a universal building block is :

(A) EX-OR gate
(B) AND gate
(C) OR gate
(D) NOR gate

हर **option** समझो:

(A) गलत: XOR (Exclusive OR) से सारे gates नहीं बन सकते (NOT और AND अकेले नहीं बनते)
(B) गलत: सिर्फ AND से NOT नहीं बन सकता
(C) गलत: सिर्फ OR से NOT नहीं बन सकता
(D) ✔ सही: NOR (NOT OR) universal gate है, इससे NOT, AND, OR सब बन जाते हैं

Ans: (D) NOR gate

Universal gate वो gate है जिससे हर दूसरा gate बन सके. ऐसे सिर्फ दो gate हैं: NAND (NOT AND) और NOR (NOT OR). इस सवाल में NOR ही option है. जवाब (D). याद: NOR(A,A) = NOT A.

Q40. [UGC NET Jun 2009]

The boolean expression x'y'z+yz+xz is equivalent to:

(A) x
(B) y
(C) z
(D) x+y+z

हर **option** समझो:

(A) गलत: अगर answer x होता तो z वाला term बाकी नहीं रहता
(B) गलत: y आता तभी जब z का गुणा न होता
(C) ✔ सही: z(x'y' + y + x) = z.1 = z, क्योंकि x'y' + x + y हमेशा 1 है
(D) गलत: x+y+z तब आता जब z का गुणा न बचता

Ans: (C) z

तीनों terms में z common है. z निकालो: z(x'y' + y + x). अब x'y' असल में (x+y)' है (De Morgan). तो अंदर (x+y)' + (x+y) बनता है, जो A' + A = 1 है. इसलिए F = z.1 = z. जवाब (C).

Step-by-step Solution:

1. दिया: F = x'y'z + yz + xz. z common लो: F = z (x'y' + y + x).
2. x + y और x'y' एक दूसरे का complement हैं (De Morgan: (x+y)' = x'y').
3. A + A' = 1, यहाँ A = (x+y). तो x'y' + y + x = (x+y) + (x+y)' = 1.
4. F = z . 1 = z. Answer (C).

Q41. [UGC NET Jun 2009]

The characteristic equation of a JK flip flop is :

(A) Qn+1 = J.Qn + K.Qn [bars missing in source]
(B) Qn+1 = J.Q'n + K'.Qn
(C) Qn+1 = Qn.J.K
(D) Qn+1 = (J+K).Qn

हर **option** समझो:

(A) गलत: यह option छपाई में bar गायब होने से बिगड़ा दिखता है. Logic से JK का सही equation option B है
(B) ✔ सही: JK flip-flop: Q(n+1) = J.Q(n)' + K'.Q(n). J=K=1 पर toggle, J=K=0 पर hold
(C) गलत: JK का equation product Q.J.K नहीं है
(D) गलत: (J+K).Q set होने की शर्त नहीं देता

Ans: (B) Qn+1 = J.Q'n + K'.Qn

JK का characteristic equation: Q(n+1) = J.Q' + K'.Q. जाँच: J=1,K=0: Q+ = Q' + Q = 1 (set). J=0,K=1: Q+ = 0 (reset). J=K=0: Q+ = Q (hold). J=K=1: Q+ = Q' (toggle). Source key (A) में bar छपे नहीं; सही equation option (B) है. जवाब (B).

Step-by-step Solution:

1. Equation: Q(n+1) = J.Q' + K'.Q.
2. J=0,K=0: Q+ = 0 + 1.Q = Q (hold).
3. J=0,K=1: Q+ = 0 + 0 = 0 (reset).

4. J=1,K=0: Q+ = Q' + Q = 1 (set).
5. J=1,K=1: Q+ = Q' + 0 = Q' (toggle). चारों cases JK की truth table से मिलते हैं.

Q42. [UGC NET Dec 2009]

The simplified form of the Boolean expression (X + Y + XY) (X + Z) is

(A) X + Y + ZX + Y
(B) XY – YZ
(C) X + YZ
(D) XZ + Y

हर **option** समझो:

(A) गलत: यह expression बाकी terms हटाने से बच नहीं पाता, simplify नहीं हुआ
(B) गलत: XY - YZ जैसी "minus" Boolean algebra में होती ही नहीं
(C) ✔ सही: X + YZ. पहले X+Y+XY = X+Y, फिर (X+Y)(X+Z) = X + YZ
(D) गलत: XZ + Y भी नहीं मिलता

Ans: (C) X + YZ

पहले पहला bracket: X + Y + XY. absorption से X + XY = X, तो = X + Y. अब (X + Y)(X + Z) = X + YZ (distributive law: A + BC = (A+B)(A+C)). जवाब (C).

Step-by-step Solution:

1. पहला bracket: X + Y + XY. X + XY = X (absorption), इसलिए = X + Y.
2. अब (X + Y)(X + Z).
3. Distributive law (उल्टा): (A + B)(A + C) = A + BC, यहाँ A = X, B = Y, C = Z.
4. Result = X + YZ. Answer (C).
5. Check: X=0,Y=1,Z=1: मूल = (0+1+0)(0+1) = 1; X + YZ = 0 + 1 = 1 ठीक.

Q43. [UGC NET Dec 2009]

The highest noise margin is offered by

(A) BICMOS
(B) TTL
(C) ECL
(D) CMOS

हर **option** समझो:

(A) गलत: BiCMOS (Bipolar + CMOS (Complementary Metal Oxide Semiconductor)) में speed और fan out अच्छे हैं, पर noise margin CMOS जितना नहीं
(B) गलत: TTL (Transistor-Transistor Logic) का noise margin लगभग 0.4 V, CMOS से कम
(C) गलत: ECL (Emitter Coupled Logic) का noise margin सबसे कम (लगभग 0.15 V)
(D) ✔ सही: CMOS (Complementary MOS (Metal Oxide Semiconductor)) का noise margin supply का लगभग 45 प्रतिशत, सबसे ज़्यादा

Ans: (D) CMOS

Noise margin यानी circuit कितना शोर (अनचाहा voltage) सह सकता है, फिर भी 0/1 सही पहचाने.

CMOS (Complementary Metal Oxide Semiconductor) में यह supply voltage का लगभग 45% होता है, इसलिए सबसे ज़्यादा. Source की key (B) TTL (Transistor-Transistor Logic) थी, पर उसी source की explanation में CMOS को "excellent noise immunity" लिखा है, इसलिए हम (D) लेते हैं.

Q44. [UGC NET Jun 2010]

What is the transitive voltage for the voltage input of a CMOS operating from 10V supply ?

(A) 1V
(B) 2V
(C) 5V
(D) 10 V

हर **option** समझो:

(A) गलत: 1 V बहुत कम है, input switching point (transition voltage) बीच में होता है
(B) गलत: 2 V भी बीच में नहीं है
(C) ✔ सही: CMOS (Complementary Metal Oxide Semiconductor) का transition voltage लगभग Vdd/2 = 10/2 = 5 V
(D) गलत: 10 V तो पूरी supply है, transition नहीं

Ans: (C) 5V

CMOS (Complementary Metal Oxide Semiconductor) inverter में output तब बदलता है जब input लगभग supply का आधा हो. Vdd = 10 V तो transition voltage = 5 V. जवाब (C).

Step-by-step Solution:

1. Formula: Transition (threshold) voltage = Vdd / 2 (Vdd = supply voltage).
2. Vdd = 10 V.
3. Transition voltage = 10 / 2 = 5 V.
4. Answer (C).

Answer Key

Q1 - B	Q2 - C	Q3 - D	Q4 - A	Q5 - B
Q6 - C	Q7 - A	Q8 - A	Q9 - B	Q10 - C
Q11 - A	Q12 - C	Q13 - B	Q14 - D	Q15 - E
Q16 - D	Q17 - C	Q18 - A	Q19 - A	Q20 - A
Q21 - D	Q22 - C	Q23 - D	Q24 - A	Q25 - D
Q26 - D	Q27 - C	Q28 - D	Q29 - D	Q30 - A
Q31 - B	Q32 - B	Q33 - A	Q34 - C	Q35 - A
Q36 - A	Q37 - C	Q38 - B	Q39 - D	Q40 - C
Q41 - B	Q42 - C	Q43 - D	Q44 - C	